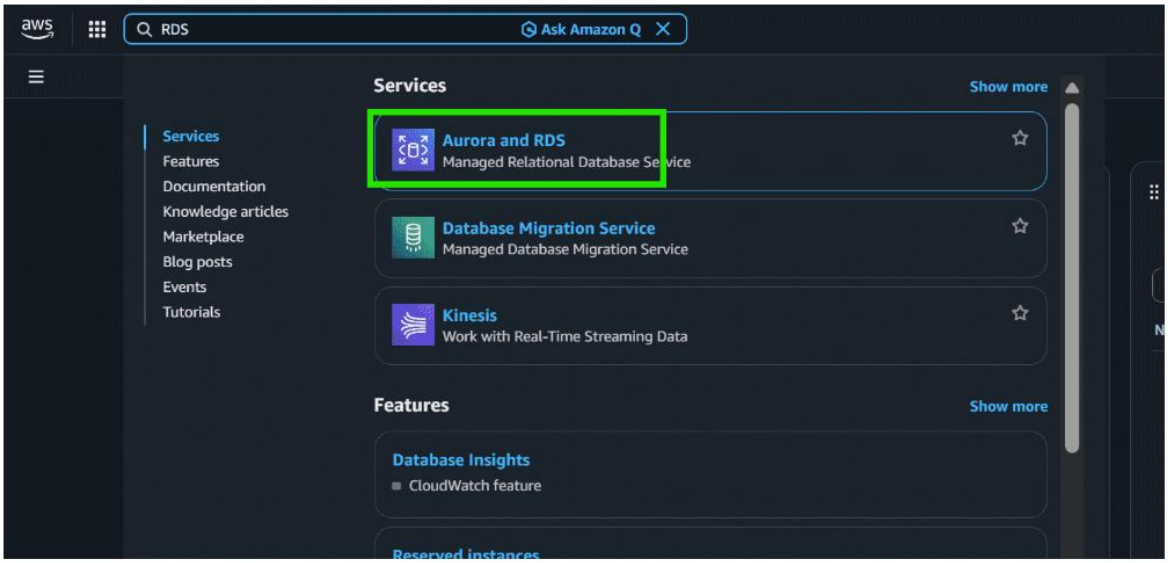
Create a Database Using AWS RDS

- Amazon Relational Database Service (RDS) is a distributed managed relational database service offered by AWS. It automates administrative tasks such as hardware provisioning, database setup, patching, and backups, allowing you to focus on your application.
  - Enables database hosting without purchasing or managing dedicated physical servers.
  - AWS handles patching, backups, and infrastructure maintenance.
  - Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora.
  - Resize compute and storage resources with minimal downtime.
  - Billed only for the resources consumed; no upfront capital investment.

Create a Database on AWS RDS

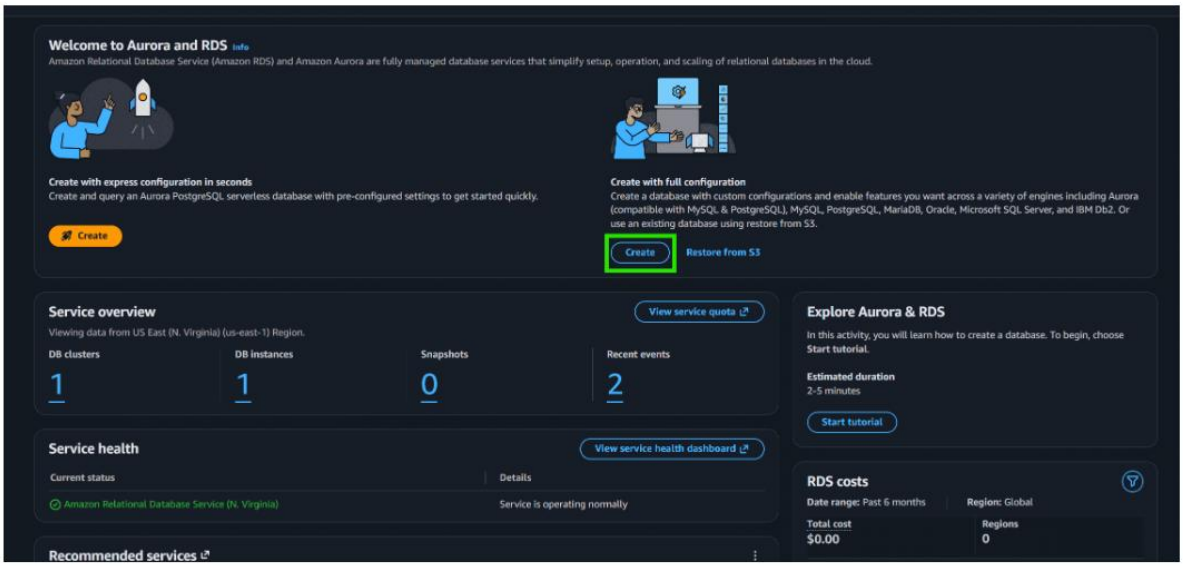
Step 1: Open the Amazon RDS Console

- Log in to your AWS account.
- Open the AWS Management Console.
- Search for RDS in the Services search bar.
- Select Amazon RDS.



Step 2: Create a Database

- Click the blue Create button under Create with full configuration.
- This opens the database creation wizard.



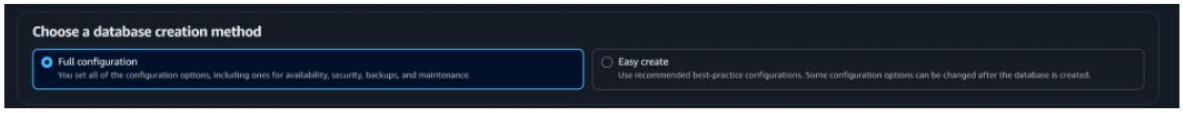
Step 3: Select Database Engine

- In the Engine options section, choose MySQL as the database engine.



Step 4: Select Database Creation Method

- Choose:
    - Easy create for automatic recommended settings, or
    - Full configuration for manual setup.
- Select Full configuration for better customization.

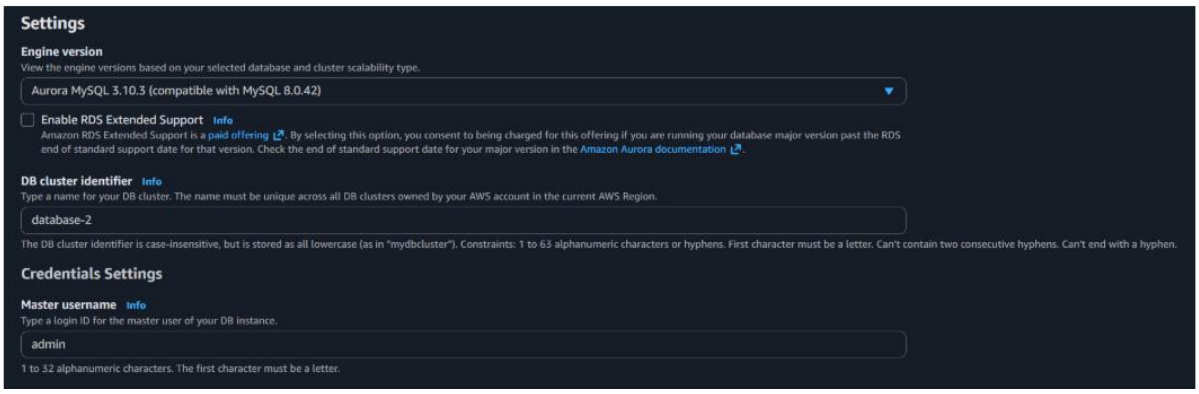


Step 5: Choose a Template

- In the Templates section, select:
    - Free Tier for learning and practice
    - Dev/Test for development environments
    - Production for live applications
- Choose Dev/Test Tier to avoid unnecessary charges while testing.

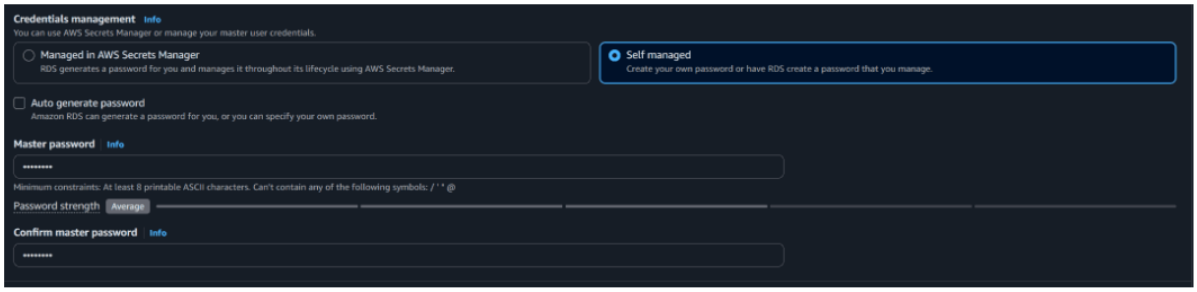
Step 6: Configure Database Settings

- Enter the following details:
    - DB Instance Identifier
    - Master Username
    - Master Password
- Example:
    - Identifier: `database-2`
    - Username: `admin`



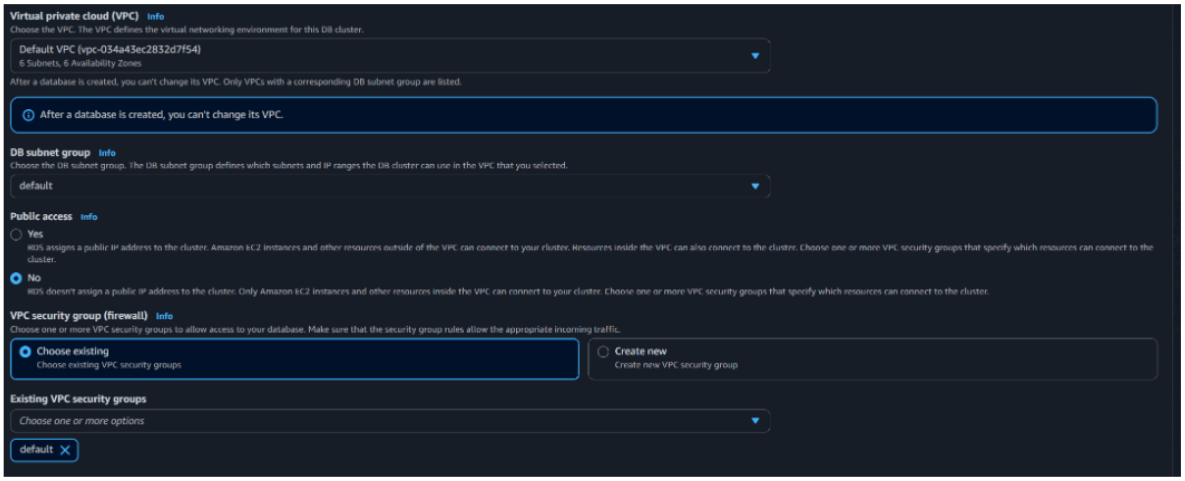
Step 7: Configure Credentials Management

- Select:
    - Self managed to create your own password, or
    - Managed in AWS Secrets Manager for automatic password management.
- Choose Self managed for simple setup and lower cost.



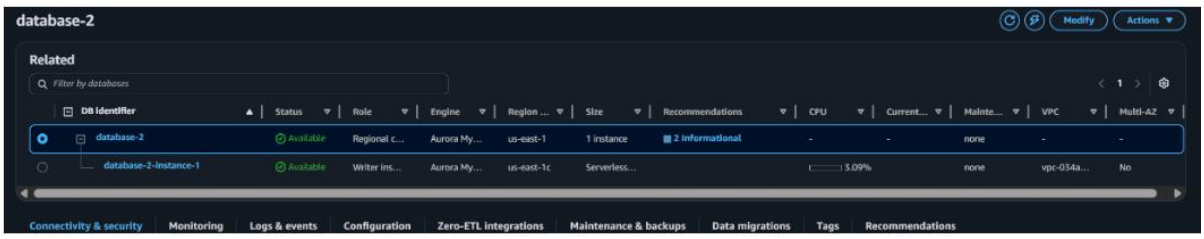
Step 8: Configure Connectivity

- In the Connectivity section:
    - Select the default VPC
    - Choose the default DB subnet group
- Choose Public Access "No"
- Select or create a VPC Security Group.



Step 9: Create the Database

- Review all configurations carefully.
- Click Create Database at the bottom of the page.
- AWS automatically starts provisioning the MySQL RDS instance.
- The database status will initially show as Creating.
- Wait until the status changes to Available.



Step 10: Connect to the Database

- Open the created database instance.
- Copy the Endpoint (DNS Name) from the Connectivity section.
- Use the endpoint, username, and password to connect through:
    - MySQL Workbench
    - DBeaver
    - Command Line Interface (CLI)

Advantages

- **Cost Efficient:** Eliminates on-premises hardware costs including power, cooling, and administration. Pay only for what you use.
- **Reliability:** AWS provides redundant infrastructure and automatic failover, making managed databases more resilient than typical on-premises setups.
- **Simplified Management:** Central administration, vendor-managed infrastructure, and SLA-backed uptime reduce operational burden.

What is DynamoDB

DynamoDB offers zero infrastructure management, instant scaling, pay-per-request billing, and no cold starts, upgrades, or maintenance windows. These capabilities, along with native vector search, make DynamoDB ideal for agentic applications that need semantic retrieval. For globally distributed applications, DynamoDB global tables is a multi-Region, multi-active database with up to 99.999% availability. It supports multi-Region strong consistency enabling your applications to always stay available and read the same data from any Region, so you can build zero recovery point objective applications. A broad set of DynamoDB security controls and compliance standards satisfy the requirements for governments, global banks, and other high-sensitivity organizations.

DynamoDB - Create Table

Tables are the backbone of any Relational Database Management System. Tables are used to store data in an organized form so operation on the data and retrieval of data becomes seamless. The AWS DynamoDB

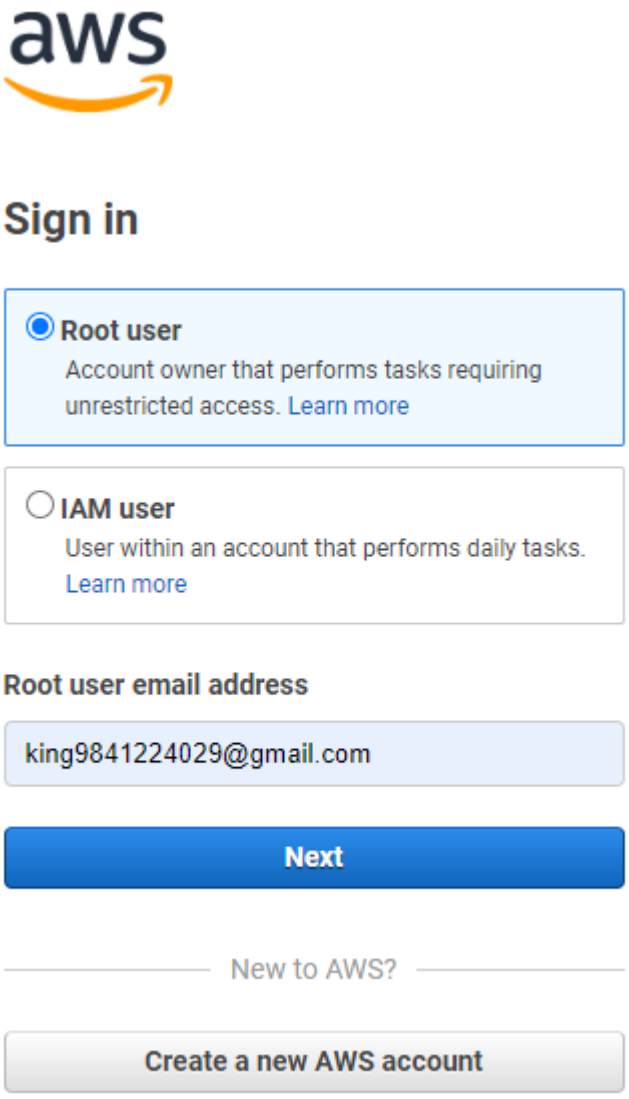
provides both RDBMS and Non-relational databases. This article mainly focuses on the relational part of DynamoDB.

In this article, we will look into the process of creating a table in DynamoDB through the AWS Management Console. After logging into the AWS Management console follow the below steps to create a table (Say, Music). The Music table will have the following details:

- Partition key: Artist
- Sort key: SongTitle

Follow the below steps to create a table in the DynamoDB database:

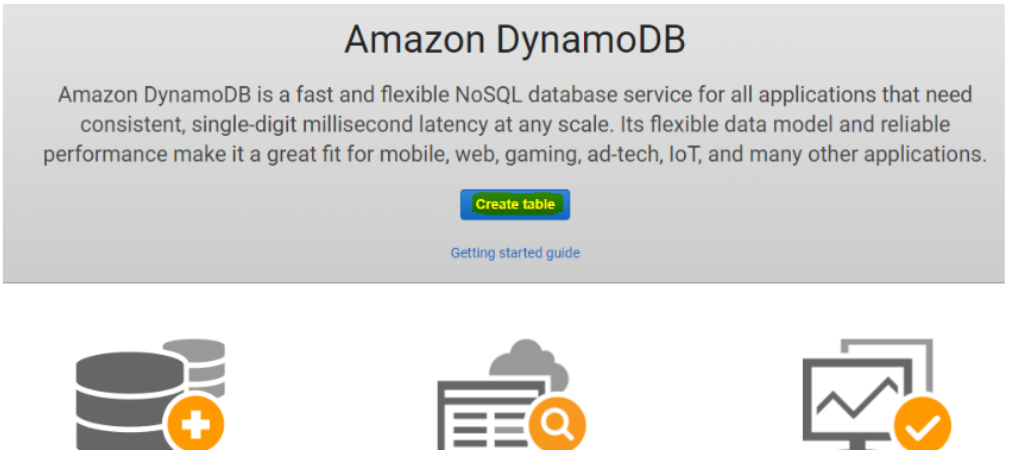
Step 1: Sign in to the AWS Management Console and open the DynamoDB console.



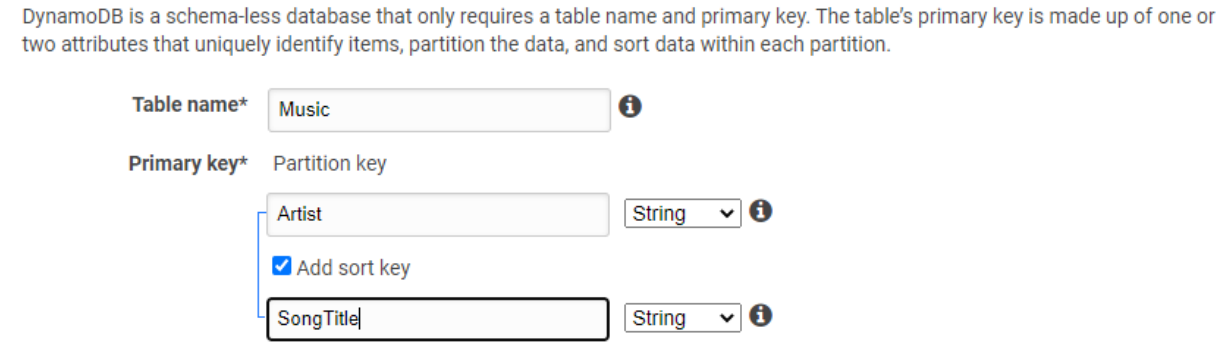
Step 2: In the navigation pane on the left side of the console, choose **Dashboard**.



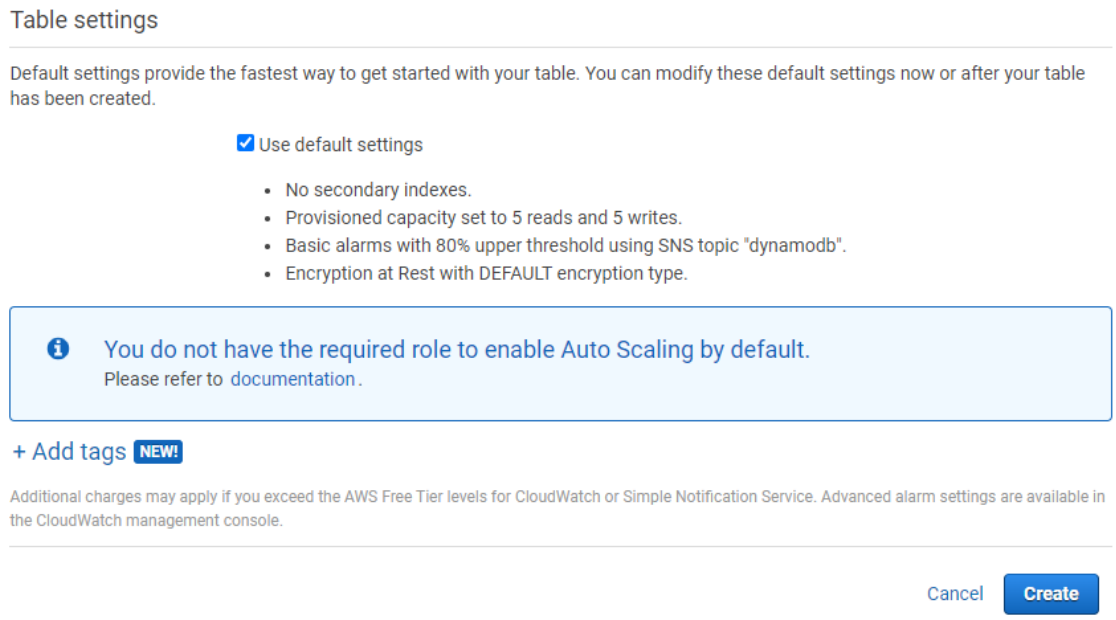
Step 3: On the right side of the console, choose Create Table.



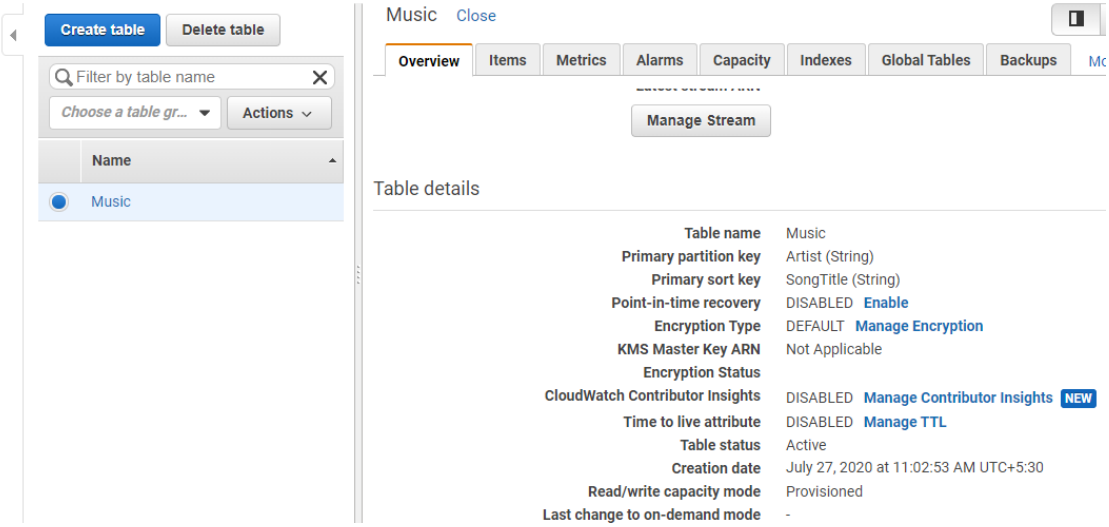
Step 4: Fill in the table details as depicted below:



Step 5: After filling in the details click on Create:



At this point your table is ready and you can start adding data into it.



***Capacity management in DynamoDB**:*

When creating a table in DynamoDB, you can specify the capacity mode for the table and its global secondary indexes. The capacity mode determines how you pay for read and write throughput for the table and its indexes. DynamoDB offers two capacity modes: on-demand and provisioned.

In on-demand capacity mode, DynamoDB automatically scales throughput capacity based on the workload demand. You pay for the actual reads and writes that you perform on the table. This can be a good option for unpredictable workloads or for getting started with DynamoDB.

In provisioned capacity mode, you specify the number of read and write capacity units that you want to allocate for the table and its indexes. You are charged for the allocated capacity units, even if you are not using them. This can be a good option for workloads with predictable or stable performance requirements.

It is important to carefully consider the capacity mode and capacity allocation for a DynamoDB table, as it can affect the cost and performance of the table. It is recommended to monitor the performance and capacity utilization of a DynamoDB table to ensure that it is configured optimally.

Amazon Redshift

Amazon Redshift is a fully managed, petabyte-scale data warehouse service in the cloud. It allows you to analyze all your data across your data warehouse and data lake using standard SQL. It is highly optimized for complex analytical queries on massive datasets.

- Fully managed, petabyte-scale cloud data warehouse designed for large-scale analytical query processing.
- Organizes data by column rather than row to significantly reduce disk I/O and boost query performance.
- Distributes query execution across multiple compute nodes to process massive datasets in parallel.
- Offers both Provisioned Cluster (with RA3 or DC2 nodes) and Serverless options to match varying workloads.
- Features tools like Redshift Spectrum to directly query exabytes of unstructured data in Amazon S3.

Core Architecture

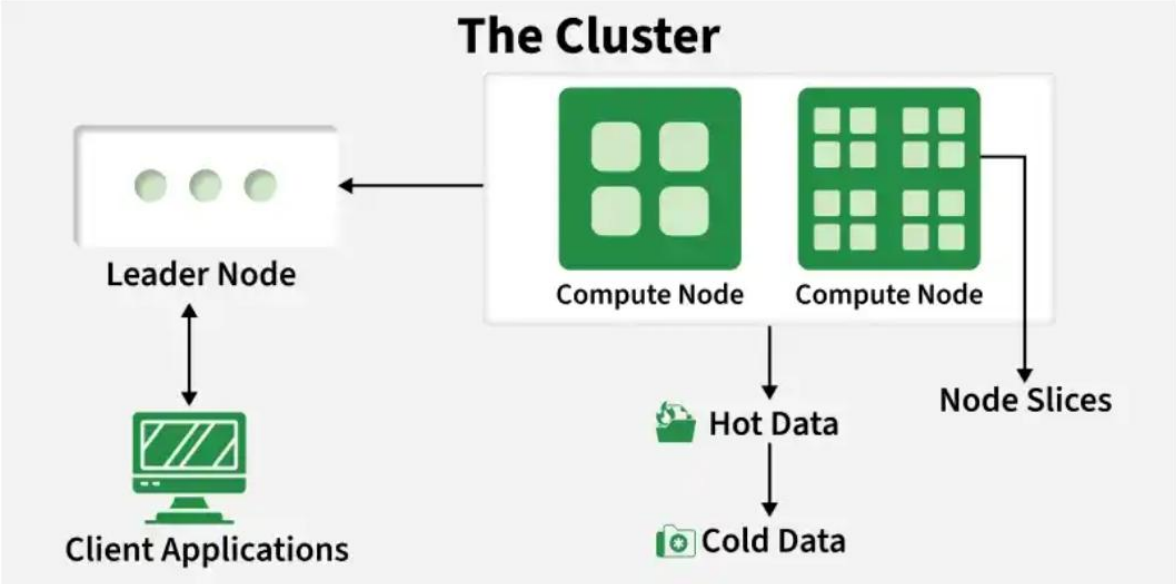
Amazon Redshift operates on a unique clustered architecture designed for distributed processing.

Cluster

- **Leader Node:** Coordinates the cluster, parses queries, creates execution plans, and stores database metadata. It does not store user database records.
- **Compute Nodes:** Store the actual user data and perform analytical queries in parallel.
- **Node Slices:** Sub-allocations of memory and disk space within compute nodes for high performance.

Redshift Managed Storage (RMS)

- Separates compute and storage scaling using RA3 nodes.
- **Hot Data:** Frequently used data stored on local SSDs for fast access.
- **Cold Data:** Less-used data automatically moved to Amazon S3 for cost-effective storage.



Key Concepts for Performance

Key Concepts for Performance

Columnar Storage

- Stores data column-wise instead of row-wise.
- Reduces disk I/O by scanning only required columns.

Distribution Styles

- **AUTO:** Redshift automatically chooses the best distribution style.
- **KEY:** Distributes data based on a selected column for faster joins.
- **EVEN:** Distributes rows evenly across all nodes.
- **ALL:** Copies the full table to every node for faster access to small tables.

Sort Keys

- Organizes data physically based on selected columns.
- Helps queries skip unnecessary data blocks and improves filtering performance.
-

Deployment Options

Amazon Redshift provides two main operational models to fit different workload types:

Amazon Redshift Provisioned

- **How it works:** Users manually configure node types (e.g., `ra3.4xlarge`) and scale the cluster size explicitly.
- **Best For:** Steady, predictable production workloads demanding highly consistent performance and precise budget control.
- **Scaling:** Scaled manually via cluster resizing or automatically via Concurrency Scaling for peak times.

Amazon Redshift Serverless

- **How it works:** Automatically provisions and scales database capacity (measured in Redshift Processing Units or RPUs) based on active query volume.
- **Best For:** Variable or intermittent workloads, ad-hoc analysis, development/testing environments, and easy operations.
- **Zero-ETL:** Supports automated, pipeline-free replication of transactional data from services like Amazon Aurora or DynamoDB.

Advanced Features

Redshift Spectrum

- **Functionality:** Enables querying structured or semi-structured data directly in S3 using standard ANSI SQL without loading.
- **Use Case:** Minimizes storage costs by keeping older, rarely-accessed historical logs in S3 and joining them with active data in Redshift.

Data Sharing

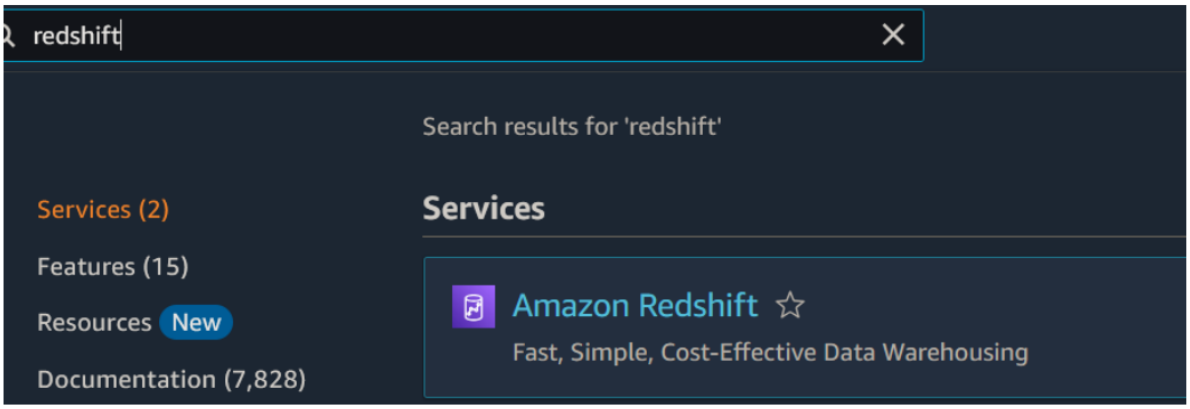
- **Functionality:** Permits sharing live, read-only data across different Redshift clusters, AWS accounts, or regions without copy overhead.
- **Use Case:** Allows isolated marketing or financial analysis workloads to query master sales data without affecting operational performance.

Set Up and Use Amazon Redshift

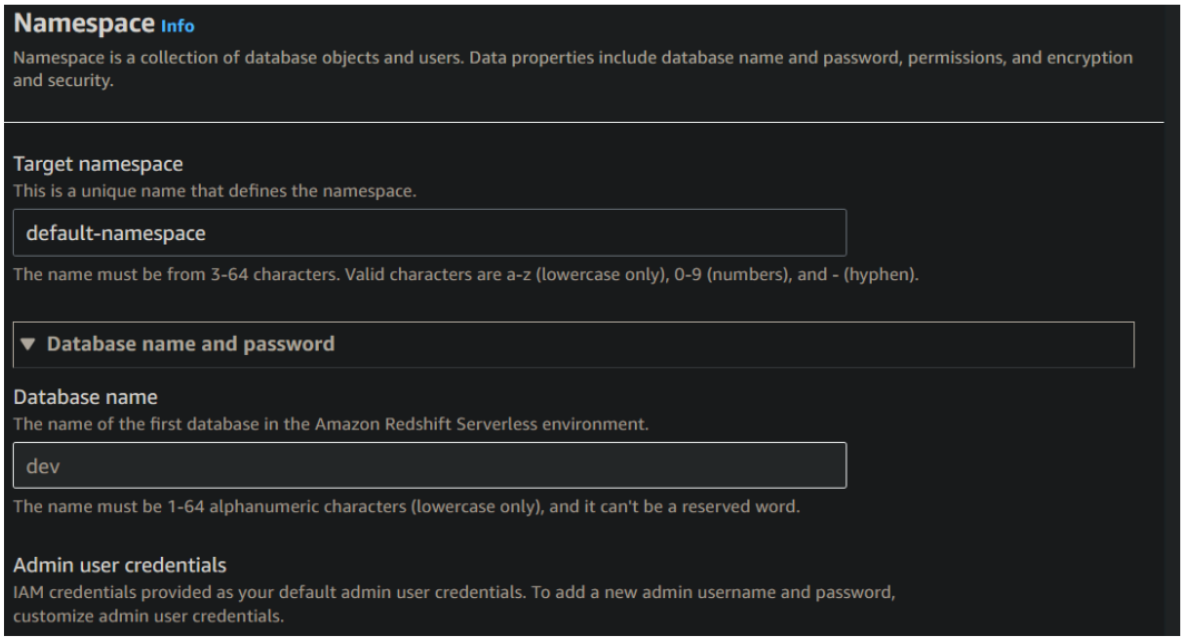
Follow these steps to configure a Redshift cluster, set up security, and load data:

Step 1: Create and Configure the Redshift Cluster

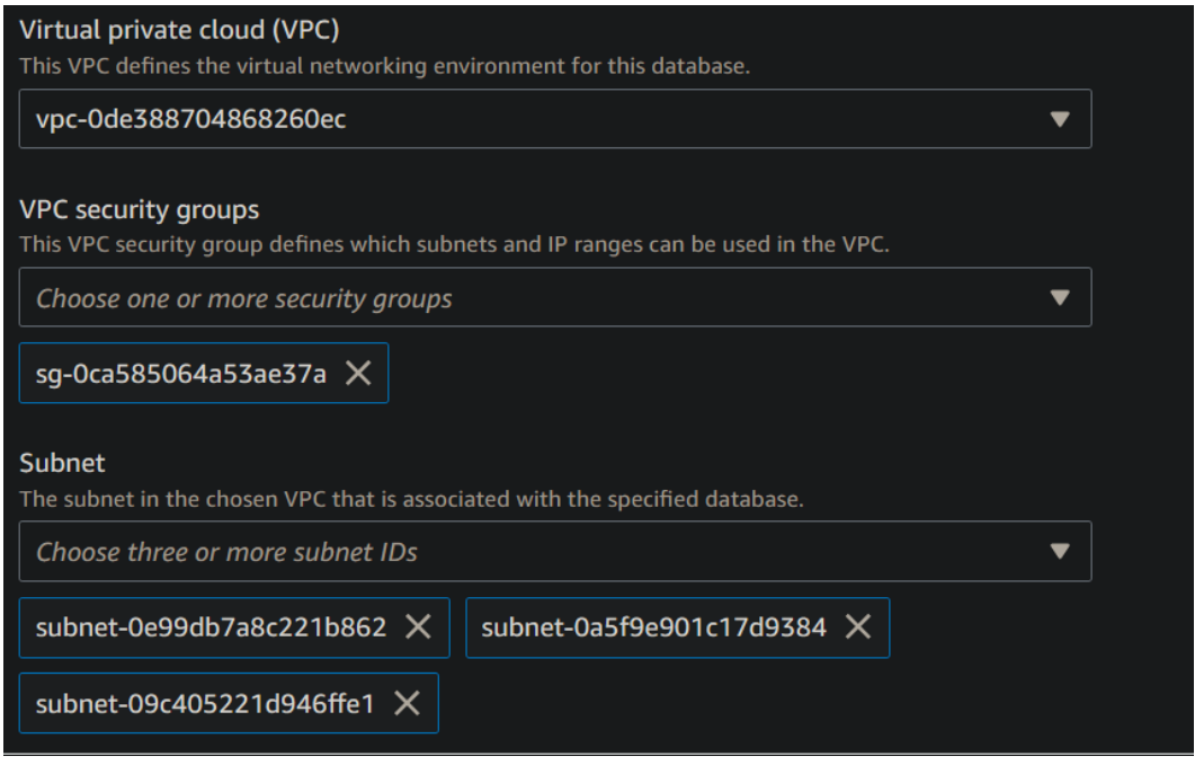
- **Navigate to Service:** Log in to the AWS Management Console, search for Amazon Redshift, and click on Create cluster.



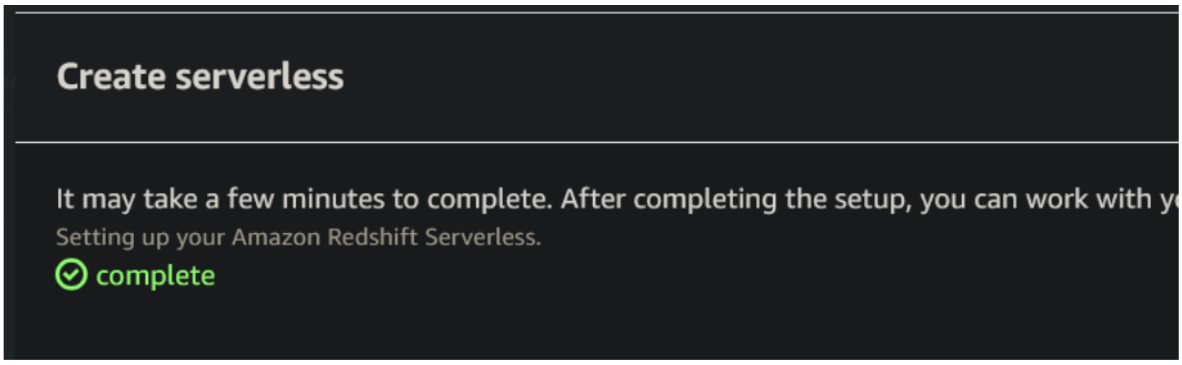
- **Configure Details:** Define the cluster identifier, master database username, database name, and admin password.



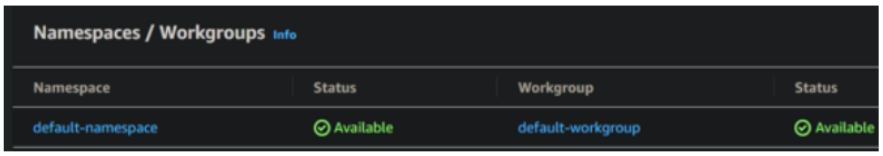
- **Set Network Options:** Choose the Virtual Private Cloud (VPC), subnet group, and associated security groups to restrict access to trusted endpoints.



- **Launch and Verify:** Click Create cluster and wait for the cluster status to change from Creating to Available



Waiting for the cluster creation process to complete



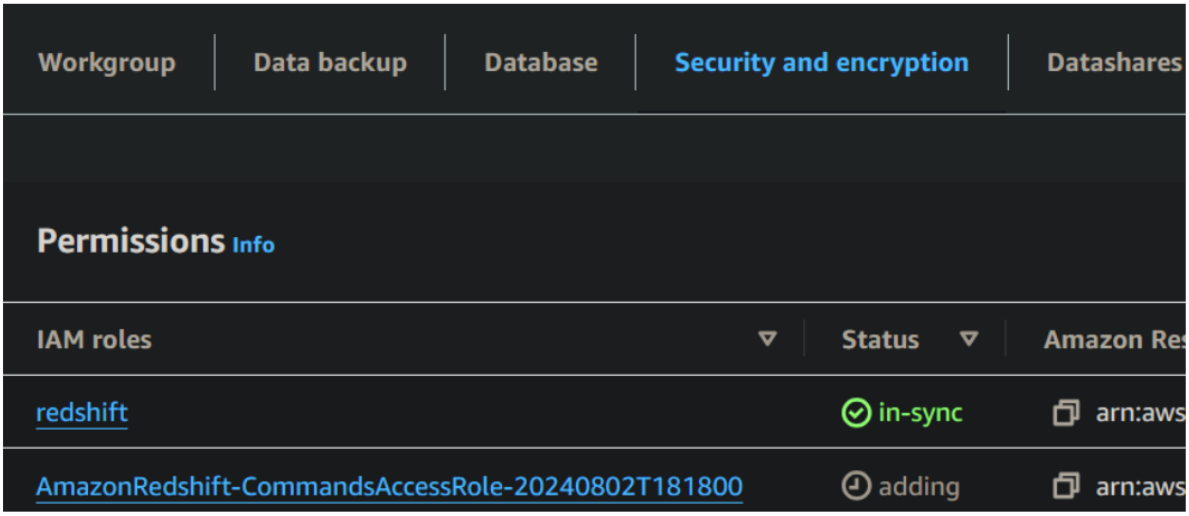
The newly created cluster is online and available

Step 2: Amazon Redshift requires IAM permissions to access services such as Amazon S3.

- **Create IAM Role:** Create an IAM role with policies that grant read permissions to S3 (e.g., AmazonS3ReadOnlyAccess).
- **Attach Role:** Navigate to the cluster's Security and encryption settings and attach the newly created IAM role.

Step 2: Amazon Redshift requires IAM permissions to access services such as Amazon S3.

- **Create IAM Role:** Create an IAM role with policies that grant read permissions to S3 (e.g., AmazonS3ReadOnlyAccess).
- **Attach Role:** Navigate to the cluster's Security and encryption settings and attach the newly created IAM role.



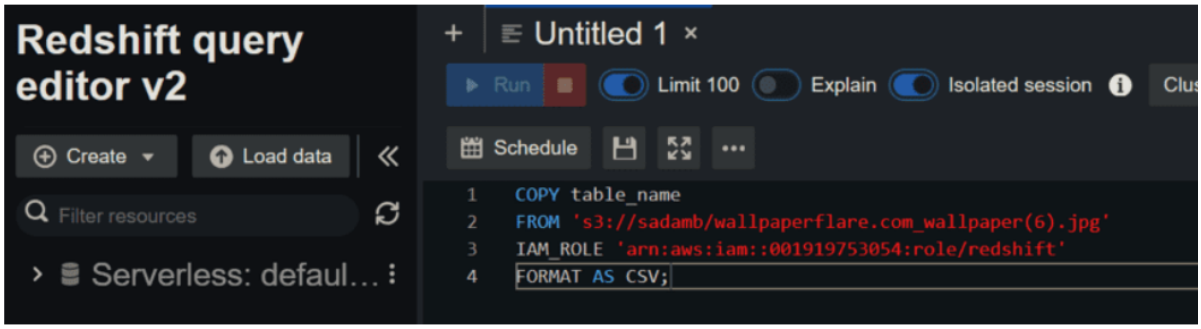
Step 3: Define your database schema by running a CREATE TABLE statement within the Query Editor:

```
CREATE TABLE sales (
    sales_id INT,
    product_name VARCHAR(255),
    quantity INT,
    price DECIMAL(10, 2),
    sale_date DATE
);
```

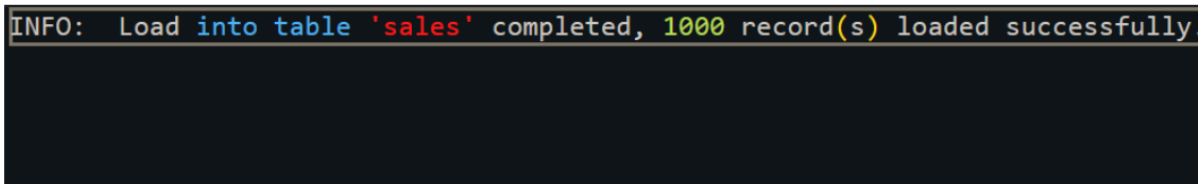
Step 4: Bulk-load data efficiently into Redshift using the COPY command:

- **Prepare Data:** Store clean, preprocessed files (e.g., CSV, Parquet) inside an Amazon S3 bucket.
- **Run COPY Command:** Execute the COPY statement to load files directly from your S3 bucket into the Redshift target table.

```
COPY sales
FROM 's3://your-bucket/your-data'
IAM_ROLE 'arn:aws:iam::your-account-id:role/your-iam-role'
FORMAT AS CSV;
```



Running the COPY command in the query editor



Benefits

- High scalability and performance
- Integration with AWS ecosystem
- Cost-effective analytics solution
- Secure data warehousing with encryption and IAM

Limitations

- Not ideal for transactional workloads (OLTP)
- Requires proper distribution and sort key design for best performance
- Query performance may vary with poor schema optimization

Pricing Models

- **On-Demand (Provisioned):** Charged a flat rate per node/hour depending on node size (e.g., approximately $3.26/hr for an ra3.4xlarge node).

- **Reserved Instances:** Offers up to 75% savings over On-Demand when committing to 1-year or 3-year terms.
- **Serverless Pricing:** Calculated per Redshift Processing Unit (RPU) per hour based on real-time query consumption.
- **Spectrum Pricing:** Billed serverlessly per Terabyte (TB) of data scanned directly in Amazon S3 (currently $5.00/TB).

AWS Cloudformation

AWS CloudFormation is an Infrastructure as Code (IaC) service that helps users create and manage AWS resources using code templates.

- It allows you to define AWS resources like EC2 instances, S3 buckets, RDS databases, and networking components in code templates.
- Automates the provisioning and deployment process, eliminating the need to manually configure resources in the AWS Console.
- Ensures infrastructure consistency by creating identical environments across multiple regions or accounts using the same template.
- Supports version control and change management, allowing you to track modifications, roll back updates, and maintain reliable infrastructure.

Templates and Stacks

CloudFormation works using two main concepts:

- **Template:** A JSON or YAML file that defines the AWS resources to be created.
- **Stack:** The collection of AWS resources created and managed from the template.

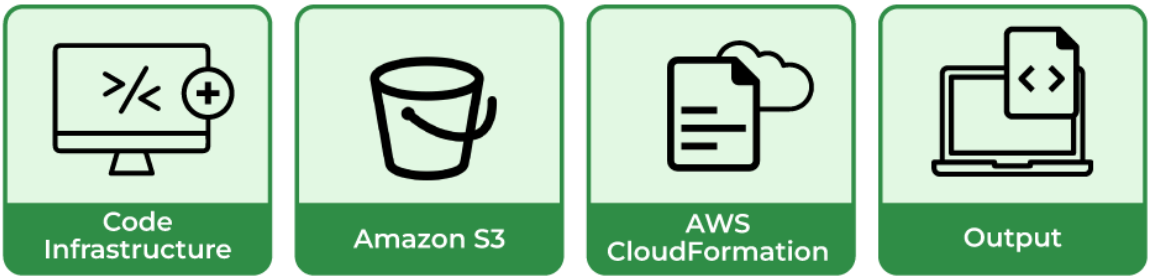
AWS CloudFormation Working

Amazon Web Services (AWS) provides cloud services such as EC2, S3, Auto Scaling, and Load Balancing. AWS CloudFormation helps automate the

provisioning of these services using Infrastructure as Code (IaC), reducing manual configuration.

Step-by-step process:

1. Define infrastructure in a template.
2. CloudFormation reads the template configuration.
3. AWS APIs are called to create resources.
4. Resources are deployed as a stack.
5. Future updates safely modify the existing stack.



Benefits

- **Automation:** Automates the creation and management of AWS resources.
- **Consistency:** Creates identical infrastructure using reusable templates.
- **Cost Savings:** Reduces manual effort and reuses existing templates.
- **Security:** Applies security policies and configurations consistently.
- **Scalability:** Easily scales resources based on demand.

Getting Started with AWS CloudFormation

CloudFormation templates are written in JSON or YAML format. In this article, we focus on the JSON template format.

JSON (JavaScript Object Notation) is a text-based format used to represent structured data. In AWS CloudFormation, JSON templates define AWS

resources and their configurations in a structured way, which CloudFormation uses to create and manage the AWS infrastructure.

Structure of CloudFormation JSON Template

- **Format Version:** Defines the version of the CloudFormation template.
- **Description:** Contains additional details or comments about the template.
- **Metadata:** Provides extra information using JSON objects.
- **Parameters:** Accepts custom or dynamic input values during stack creation.
- **Mappings:** Maps keys to corresponding values for conditional use.
- **Conditions:** Controls whether specific resources or properties are created.
- **Transform:** Helps reuse template components using declarative macros.
- **Resources:** Defines AWS resources like EC2, S3, and Lambda in the stack.
- **Outputs:** Displays values generated after stack creation, such as IDs or URLs.

Template Terms and Concepts

Understanding The Core Concepts That CloudFormation templates use to organize resources, settings, and functions is key to managing AWS infrastructure efficiently.

- **Template:** A CloudFormation template is a JSON or YAML file that defines the AWS resources and configurations to be created.
- **Stacks:** A stack is the collection of AWS resources created from a CloudFormation template and managed as a single unit.
- **Formatting:** CloudFormation templates use JSON or YAML format. YAML is commonly preferred because it is simpler and more readable.

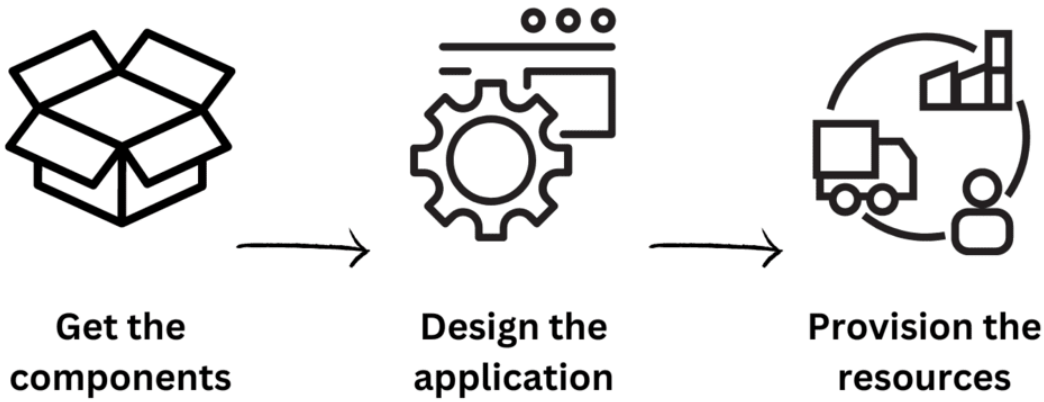
- **Change Sets:** Change Sets allow you to preview the changes CloudFormation will make before updating a stack, helping reduce deployment risks.
- **Functions:** CloudFormation provides built-in functions such as `Fn::Sub` and `Fn::Join` to create dynamic and flexible configurations.
- **Parameters:** Parameters allow users to provide custom input values during stack deployment, making templates reusable and flexible.
- **Conditions:** Conditions control whether specific resources are created or configured based on certain criteria, such as development or production environments.

AWS Cloud Development Kit

AWS CDK or Cloud Development Kit is a framework that allows you to use reusable components called constructs to build cloud environments. Developers can build and create cloud services using Integrated Development Environments with the help of libraries and components. AWS CDK provides a simple and easy approach to building infrastructure with the help of programming languages. So let's learn more about AWS CDK.

- AWS CDK includes a Command Line Interface or Toolkit that defines the design and implementation of of application where the design and implementation are written in different programming languages such as JavaScript, typescript etc.
- AWS CDK requires the definition of one or more stacks such as a cloud formation stack.
- Stack uses the construct defined above. Constructs are programming language component that contains the exact definition and structure of AWS resources.
- CDK converts this construct into cloud formation templates and then deploys or provisions the requested resources minimizing the need for complex operations.

- CDK converts this construct into cloud formation templates and then deploys or provisions the requested resources minimizing the need for complex operations.



Primary Terminologies of AWS CDK

- **Constructs:** Constructs are components containing configuration, bootstrap code, and logic for running any of the AWS Services
- **Infrastructure As A Code (IaaC)**: Infrastructure as a Code is a model or framework that provides the ability to provision and build a cloud environment with the help of code.
- **Cloud Formation:** IaaC service provided by AWS for provisioning AWS resources using code.

Benefits of AWS Cloud Development Kit

- **Reusability:** Constructs in CDK can be used multiple times to create same type of resources. Once written in a programming language construct can be reused as required.
- **Language Support:** AWS CDK provides support for multiple languages including JavaScript, Typescript, Python, Java and many more.
- **IDE Usage:** Developer can use IDE for both programming as well infrastructure management. Same code can be used from the IDE.

- **Integration of other Services:** Cloud formation integration in AWS CDK provides users with benefits of cloud formation allowing stack management, templating etc.
- **Updates and Maintain:** With the help of cloud formation the infrastructure can be updated easily.

Benefits of AWS Cloud Development Kit

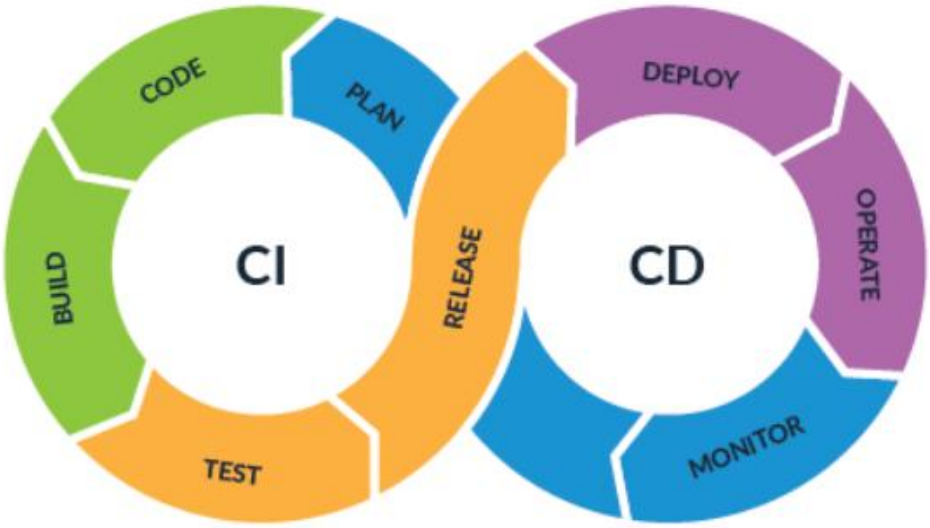
- **Reusability:** Constructs in CDK can be used multiple times to create same type of resources. Once written in a programming language construct can be reused as required.
- **Language Support:** AWS CDK provides support for multiple languages including JavaScript, Typescript, Python, Java and many more.
- **IDE Usage:** Developer can use IDE for both programming as well infrastructure management. Same code can be used from the IDE.
- **Integration of other Services:** Cloud formation integration in AWS CDK provides users with benefits of cloud formation allowing stack management, templating etc.
- **Updates and Maintain:** With the help of cloud formation the infrastructure can be updated easily.

Implementing Continuous Integration and Deployment (CI/CD) With AWS Code Pipeline

In the Rapid field of software development, implementation of Continuous Integration and Deployment (CI/CD) is essential for dependable and effective applications. This Article explores the streamlined process of setting up CI/CD using the AWS Code pipeline automating the build, test, and deployment phases. This user-friendly solution from Amazon web services(AWS) can enhance your development workflow rapidly and error-free releases.

What Is CI/CD On AWS?

When new code is submitted on one end and tested over a series of stages (source, build, test, staging, and production), and then published as production-ready code, CI/CD pipeline is used in that case.



Advantage of using CI/CD

- **Frequent releases**: Based on customer feedback or monitoring input, we can release the changes with CI/CD.
- **Low risk**: There won't be any manual intervention and configuration setup, since its automated process.
- **Increased Productivity**: Increased developer productivity, as product will be released as different components.

AWS CI/CD Tools and Services

AWS CodeCommit

- AWS CodeCommit is a secure, highly scalable, fully managed source control service that hosts private Git repositories.

- No need for the user to know Git and manage their own source control system and worry about scaling up or down their infrastructure.Even about provisioning hardware, server, scaling or software which must be installed, configured or updated.
- Codecommit repositories are encrypted so its secured.
- The size of files or file types doesn't bother file storage in Codecommit.

AWS Code Build

- AWS Code Build is a fully managed build service in the cloud. It compiles source code, runs unit tests, and produces artifacts that are ready to deploy.
- Code Build eliminates the need to provision, manage, and scale our own build servers.
- It is On demand which scales on demand to meet our build needs. We pay only for the number of build minutes we consume.
- Code Build provides preconfigured build environments for the most popular programming languages which points to our build script to start our first build.

AWS Code Deploy

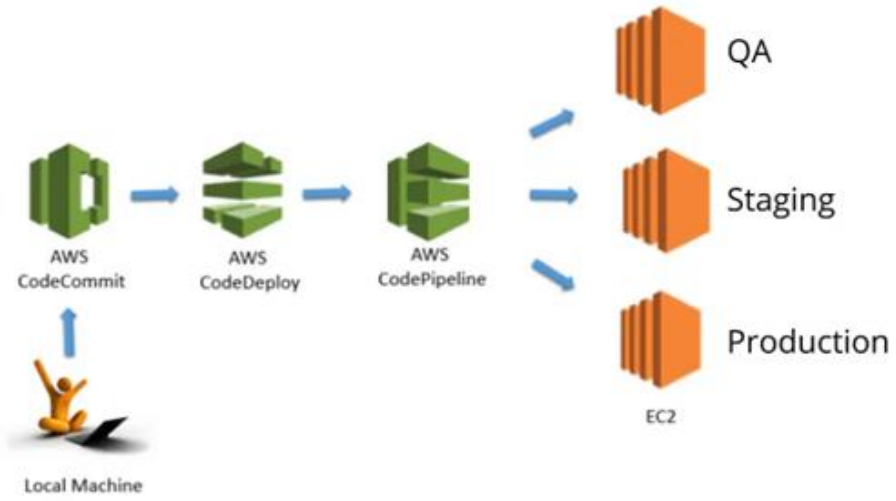
- AWS CodeDeploy is a service that automates software deployments that can deliver new features more quickly, avoid application deployment downtime, and handle the complexity of application updates with AWS CodeDeploy.
- We can easily roll out new features, prevent downtime when deploying our apps, and handle the complexity of updating our programs without many of the hazards involved with risky manual deployments.

Amazon Elastic Container Registry

- Amazon Elastic Container Registry (Amazon ECR) is a fully managed container registry which offers high-performance hosting, so we can reliably deploy application images and artifacts anywhere.
- It provides a command-line interface (CLI) and APIs to manage repositories and integrated services, such as Amazon Elastic Container Service (Amazon ECS), which installs and manages the infrastructure for these containers.
- Difference between Amazon ECR and AWS ECS is that while ECR provides the repository that stores all code that has been written and packaged as a Docker image, the ECS takes these files and actively uses them in the deployment of applications.

AWS CodeStar

- AWS CodeStar project can be used to create and integrate Amazon services with the user's project development toolchain. Based on the CodeStar project template chosen by the user, the toolchain will include source control, build, deployment, virtual servers or server less resources and other resources.
- It can be used to set up application development in the cloud and to manage the user's application with the help of a single, centralized dashboard.



What is AWS CodePipeline?

AWS CodePipeline is a fully managed continuous delivery service that helps us automate our release pipelines for fast and reliable application and infrastructure updates.

- We will create the pipeline using AWS CodePipeline, a service that builds, tests, and deploys our code every time there is a code change. We will use our GitHub account, an Amazon Simple Storage Service (Amazon S3) bucket, or an AWS CodeCommit repository as the source location for the sample app's code. We will also use AWS Elastic Beanstalk as the deployment target for the sample app. Our completed pipeline will be able to detect changes made to the source repository containing the sample app and then automatically update our live sample app.

CodePipeline Concepts

1. Pipeline: A pipeline defines release process workflow that describes how a new code change progresses through release process.
2. Stage: Stages (e.g., build, test, and deploy) act as logical divisions in workflow.
3. Revision: A revision is a change made to the source location defined on pipeline.

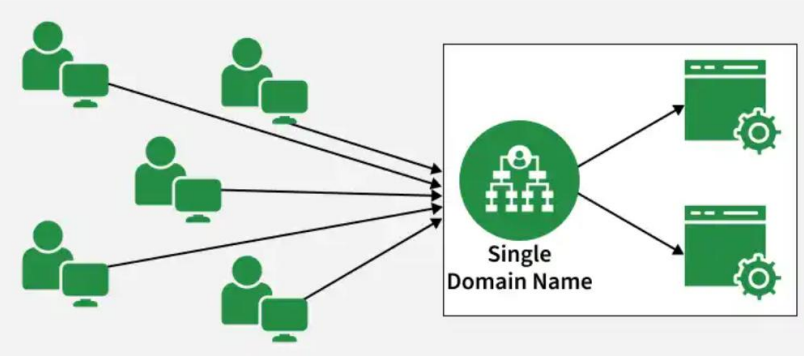
4. Action: An action is a task performed on a revision.

Elastic Load Balancer in AWS

Elastic Load Balancing (ELB) automatically distributes incoming application traffic across multiple targets, such as Amazon EC2 instances, containers, IP addresses, and Lambda functions. It acts as the "front door" for your application, ensuring high availability, fault tolerance, and elasticity.

Core Concepts

- **Listeners:** A listener checks for incoming connection requests on a specific protocol and port.
- **Target Groups:** A logical group of backend resources such as EC2 instances, containers, IP addresses, or Lambda functions. The load balancer sends traffic to the target group instead of directly to individual instances.
- **Health Checks:** ELB regularly checks the health of targets using a path like /health. If a target becomes unhealthy, ELB stops routing traffic to it until it recovers.
- **Cross-Zone Load Balancing:** Distributes traffic evenly across healthy targets in all enabled Availability Zones, helping prevent overload in a single zone.



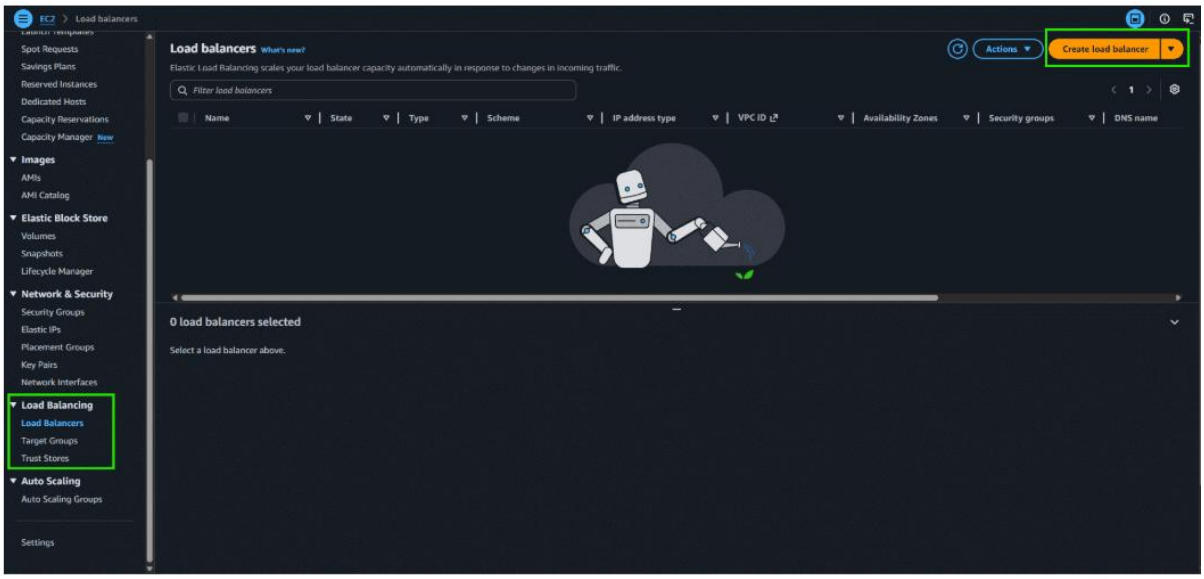
Steps to configure an Application Load Balancer in AWS

Step 1: Launch the two instances on the AWS management console named Instance A and Instance B. Go to services and select the load balancer. To

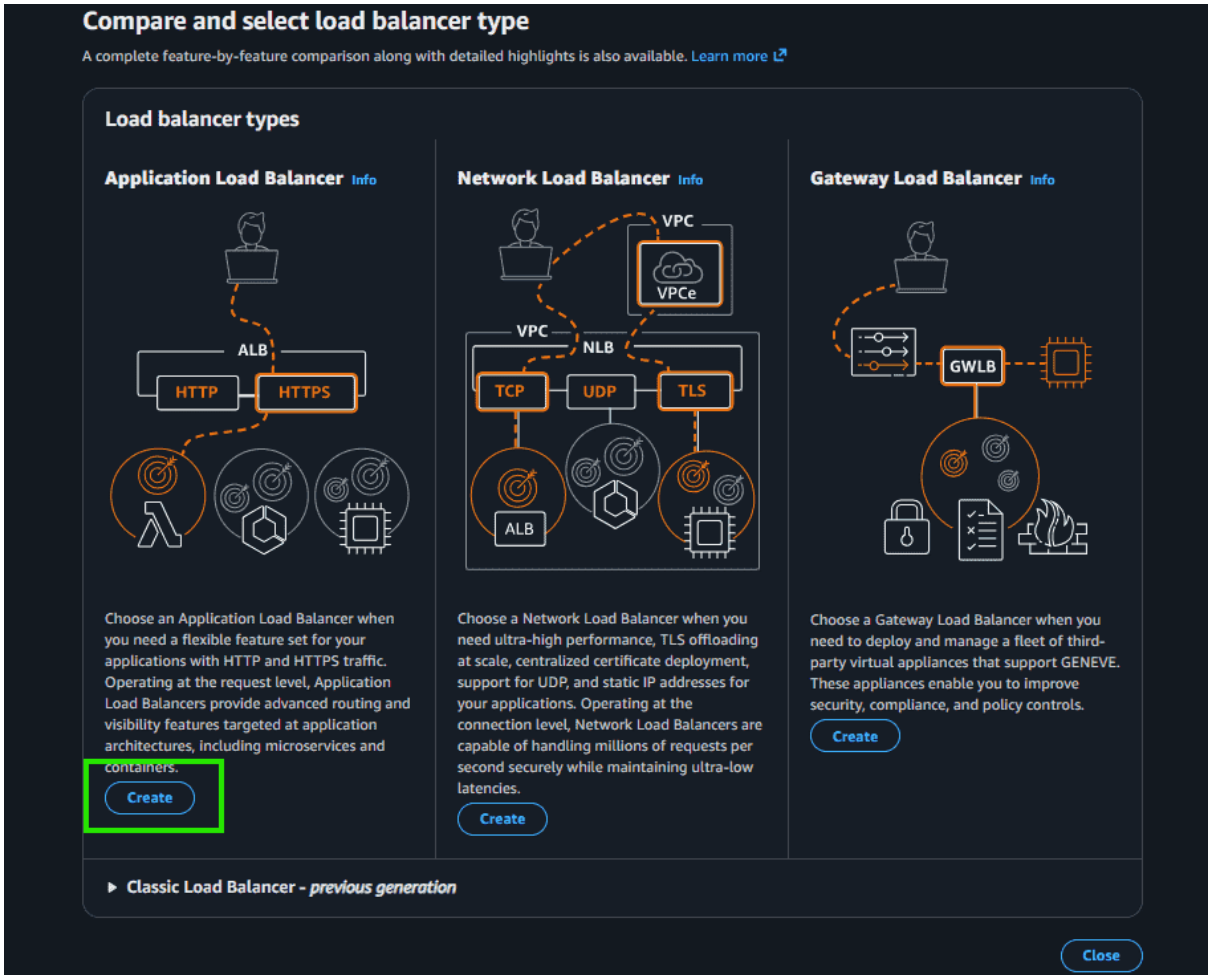
create AWS free tier account refer to [Amazon Web Services (AWS) – Free Tier Account Set up.]



Step 2: Scroll down on the left side panel, go to Load Balancers, and click on Create Load Balancer.



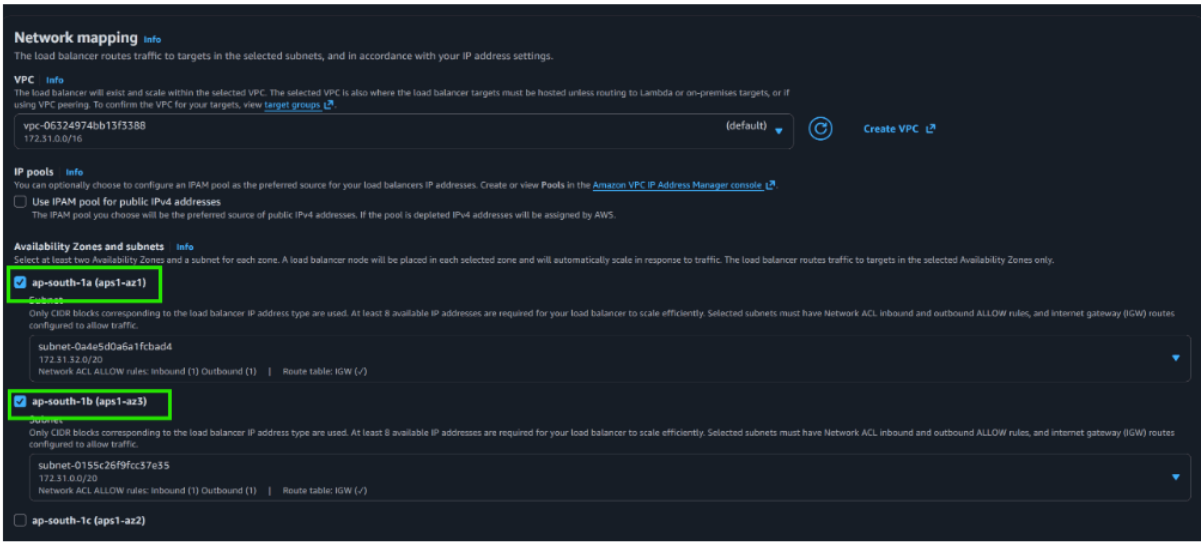
Step 3: Select Application Load Balancer and click on Create.



Step 4: Here you are required to configure the load balancer. Write the name of the load balancer. Choose the scheme as internet facing.

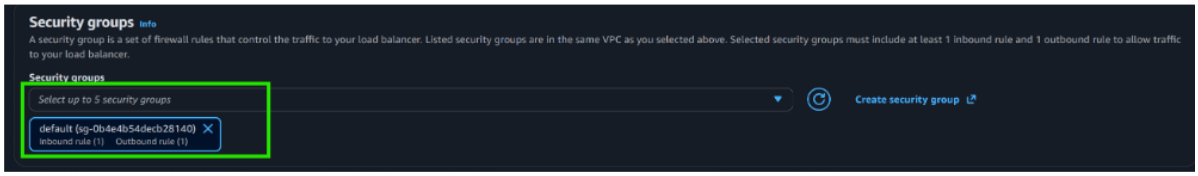


Step 5: Add at least 2 availability zones. Select us-east-1a and us-east-1b

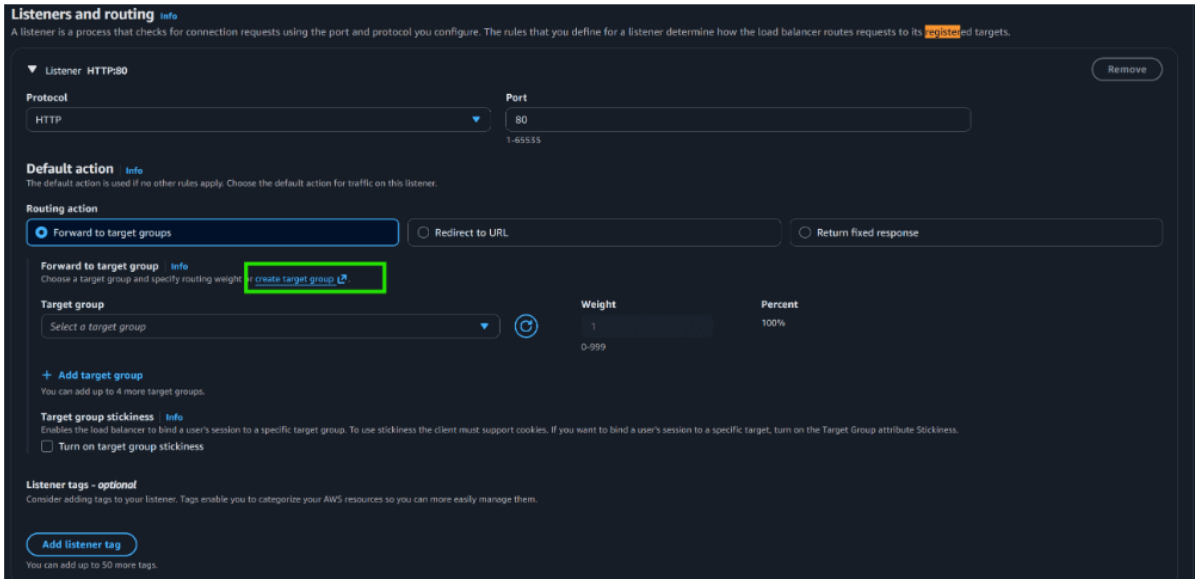


Step 6: Select the default security group.

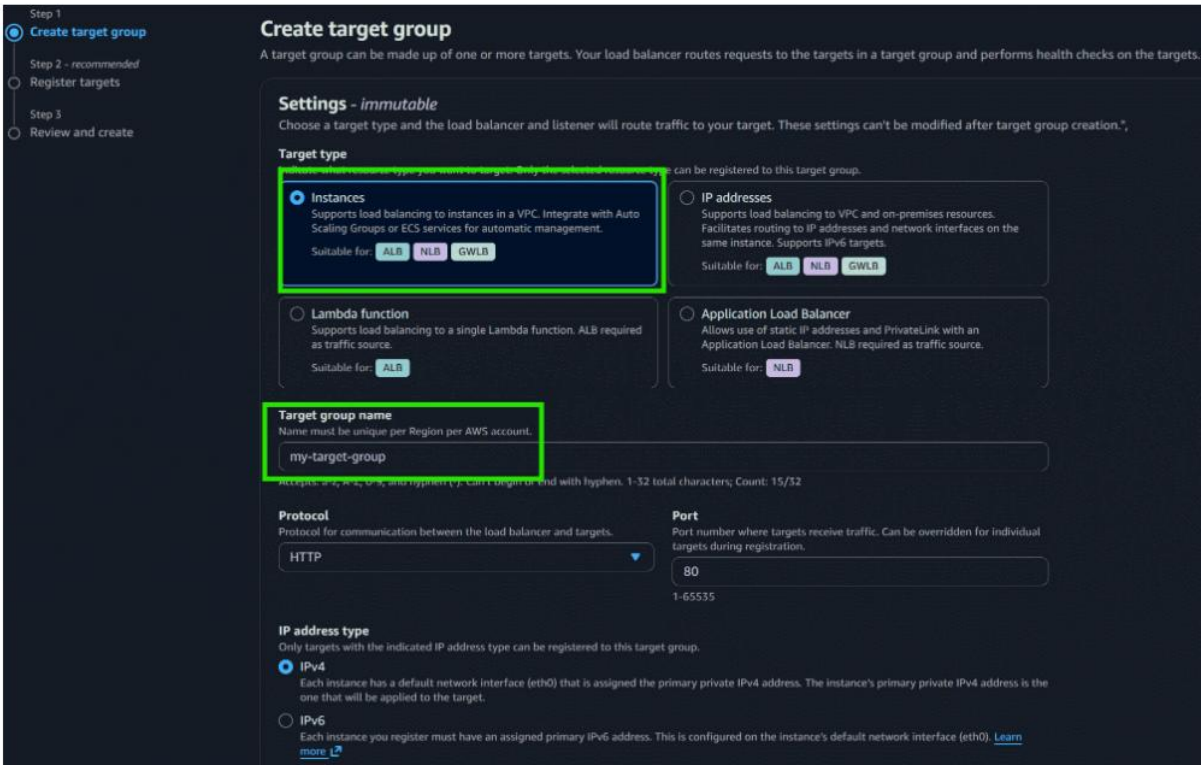
Step 6: Select the default security group.



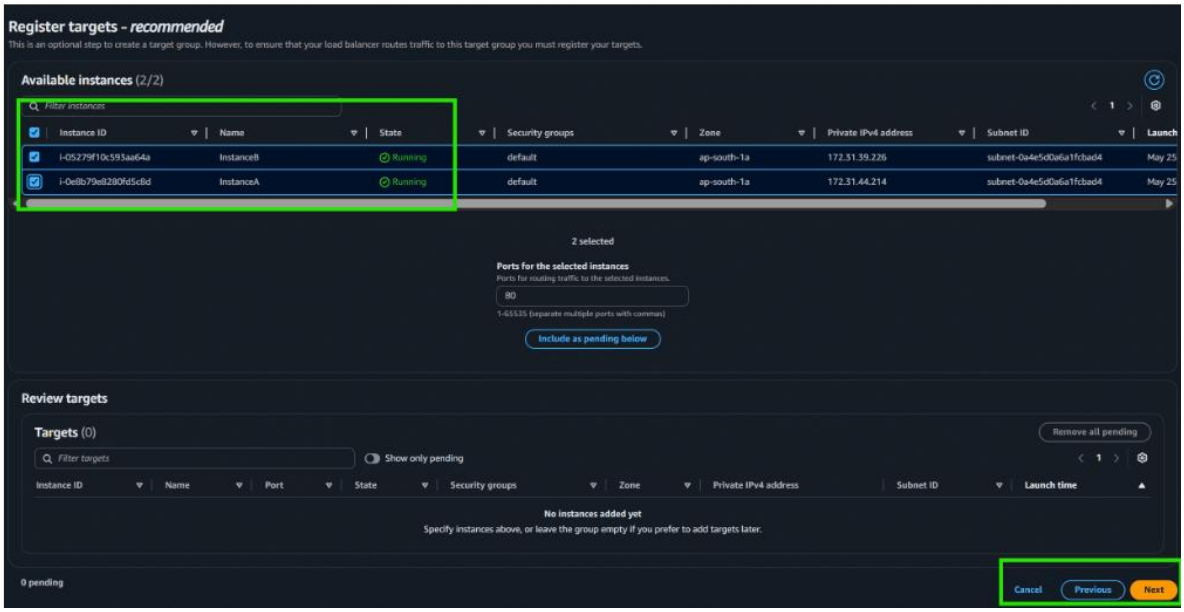
Step 7: In the Listener and Routing section, click on Create New Target Group.



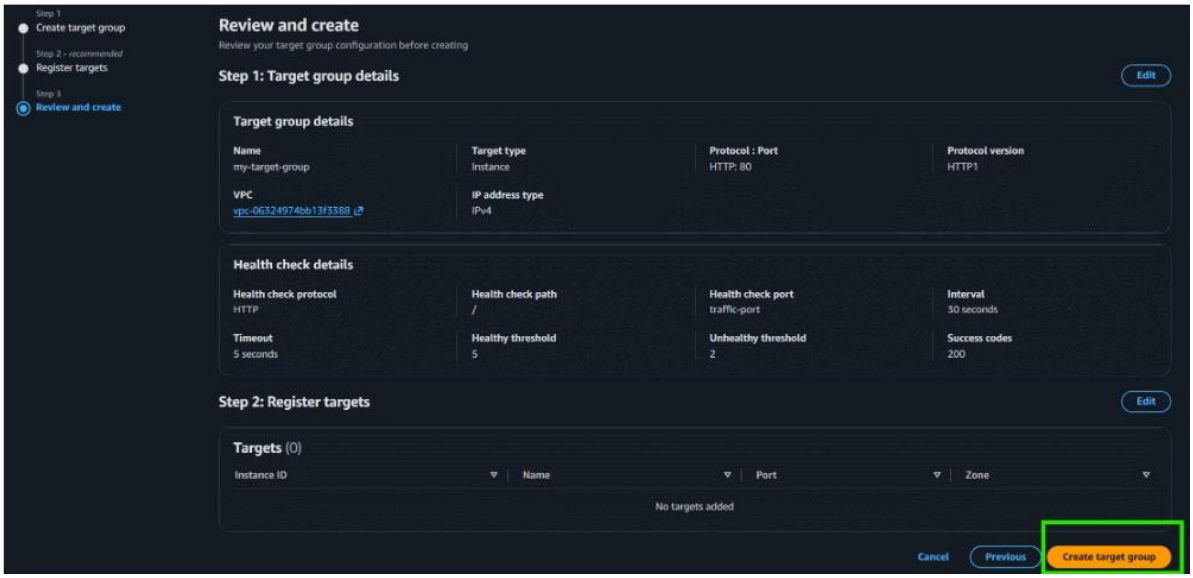
Step 8: Choose the Target Group Name, select the Protocol, and enter the Port Number.



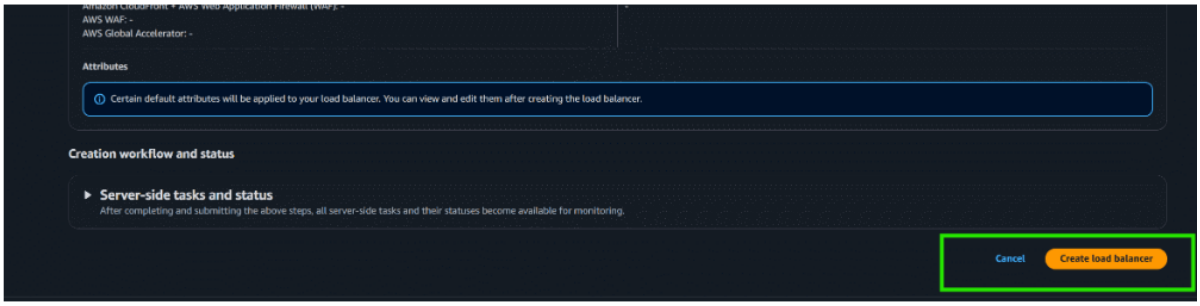
Step 9: Choose instance A and instance B and Click on Next: Review.



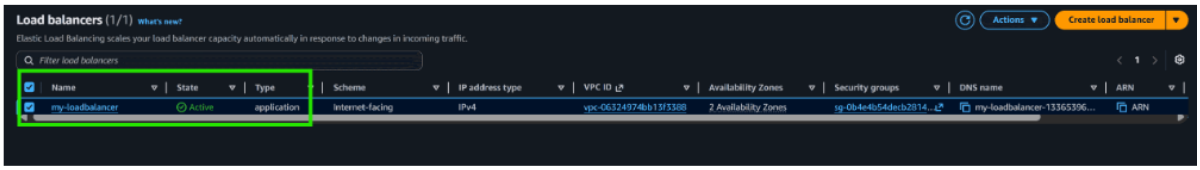
Step 10: Review all the configurations and click on Create Target Group. After that, return to the Load Balancer Creation page and select my-target-group in the routing section.



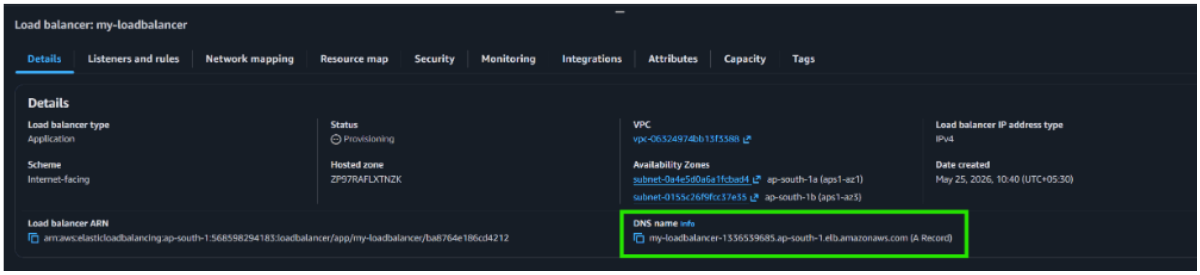
Step 11: Scroll down and click on Create Load Balancer.



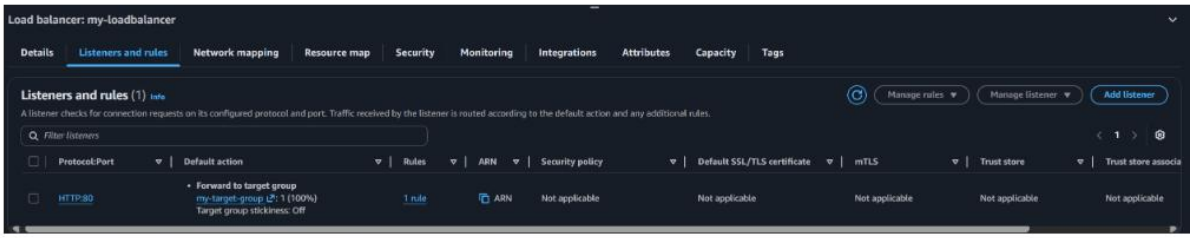
Step 12: Congratulations!! You have successfully created a load balancer.



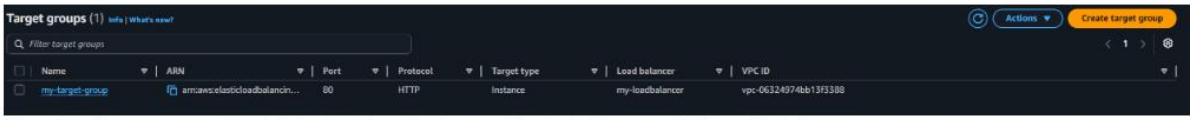
Step 13: This highlighted part is the DNS name which when copied in the URL will host the application and will distribute the incoming traffic efficiently between the two instances.



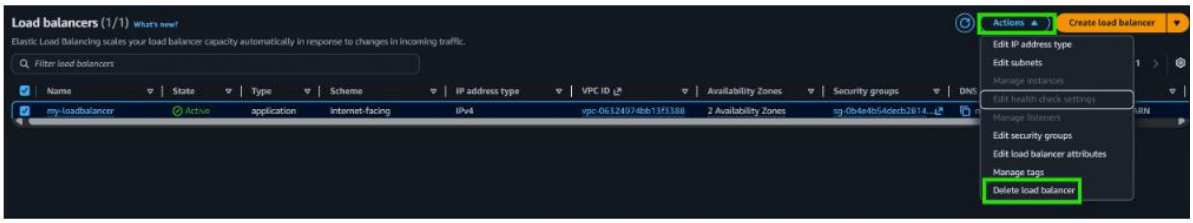
Step 14: This is the listener port 80 which listens to all the incoming requests



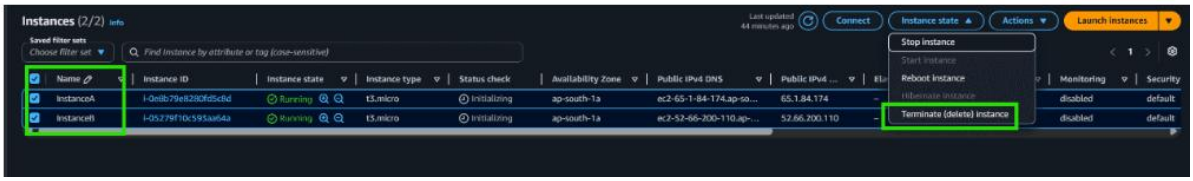
Step 15: This is the target group that we have created



Step 16: Now we need to delete the Load balancer. Go to Actions -> Click on Delete.



Step 17: Also don't forget to terminate the instances.



When to Use Which?

ALB

- Use for web applications, websites, APIs, and microservices.
- Supports HTTP, HTTPS, and gRPC traffic.
- Best when you need advanced routing based on URL paths, hostnames, or headers.
- Suitable for applications that require Layer 7 (Application Layer) features.
- Does not provide static IP addresses; access is through DNS.

NLB

- Use for gaming, financial applications, IoT systems, and real-time services.
- Supports TCP, UDP, and TLS traffic.
- Best when you need ultra-low latency and high performance.

- Provides static IP addresses for each Availability Zone.
- Suitable for applications that require Layer 4 (Transport Layer) load balancing with simple routing.

Pricing Models

ELB pricing is based on two components: an hourly charge and a usage charge (Capacity Units).

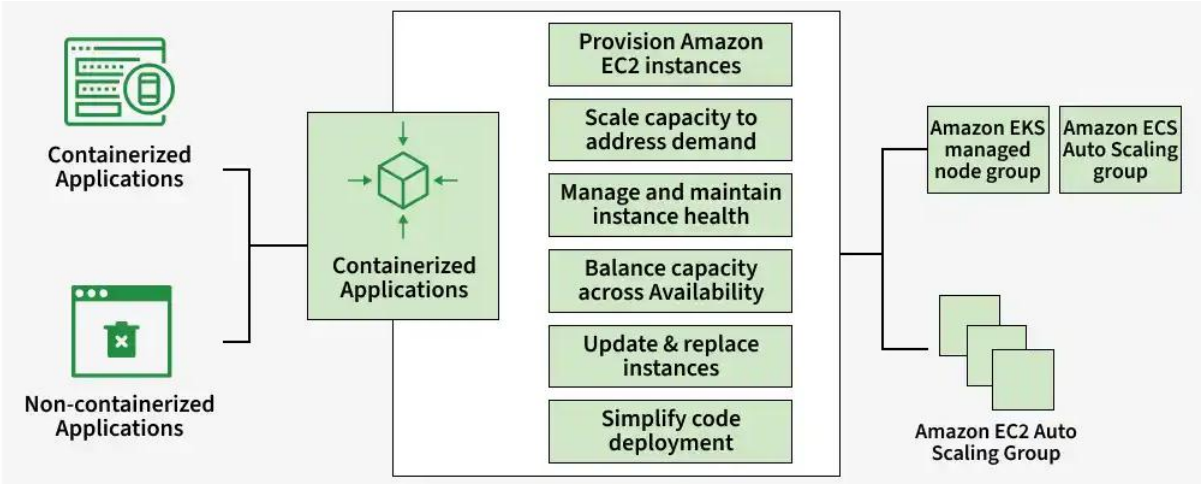
Load Balancer	Hourly Charge	Usage Metric
ALB	~$0.0225 / hour	LCU (Load Balancer Capacity Units) - Based on new connections, active connections, and bandwidth.
NLB	~$0.0225 / hour	NLCU (Network Load Balancer Capacity Units) - Based on bandwidth and connections.
GWLB	~$0.0125 / hour	GLCU (Gateway Load Balancer Capacity Units).
CLB	~$0.025 / hour	Data Transfer (GB).

Amazon EC2 Auto Scaling

Amazon EC2 Auto Scaling is a service that helps you maintain application availability and allows you to automatically add or remove EC2 instances according to conditions you define. You can use the fleet management features of EC2 Auto Scaling to maintain the health and availability of your fleet.

Instead of guessing how many servers you need (Provisioning for peak load and wasting money during low traffic periods.), Auto Scaling ensures you have exactly the right amount of compute power right when you need it.

- Amazon EC2 Auto Scaling automatically scales EC2 instances based on traffic demand while maintaining high availability and optimizing cost.
- It creates an Auto Scaling Group (ASG) where instances are managed automatically, and the Load Balancer distributes traffic among them.
- Scaling policies use metrics like CPU utilization or memory usage to automatically add or remove instances based on demand.



Core Components of Auto Scaling
To configure Auto Scaling, you need to define three main components:

1. Launch Template

- A Launch Template defines the configurations of an EC2 instance. It includes the AMI (OS image), Instance Type, Key Pair, Security Groups, and User Data used during instance launch.

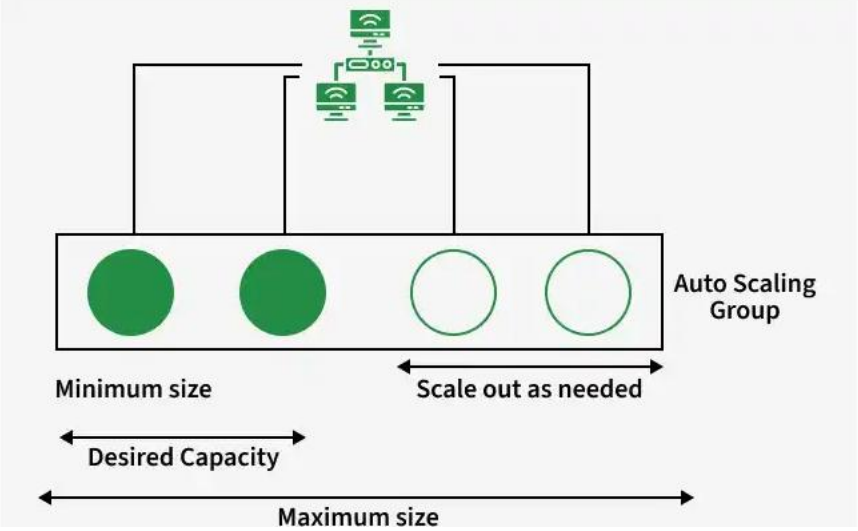
2. Auto Scaling Group (ASG)

An Auto Scaling Group creates and manages a logical group of EC2 instances. It defines the VPC and subnets where the instances will launch.

It also manages capacity limits

- **Minimum Capacity:** Minimum number of instances that must always run.

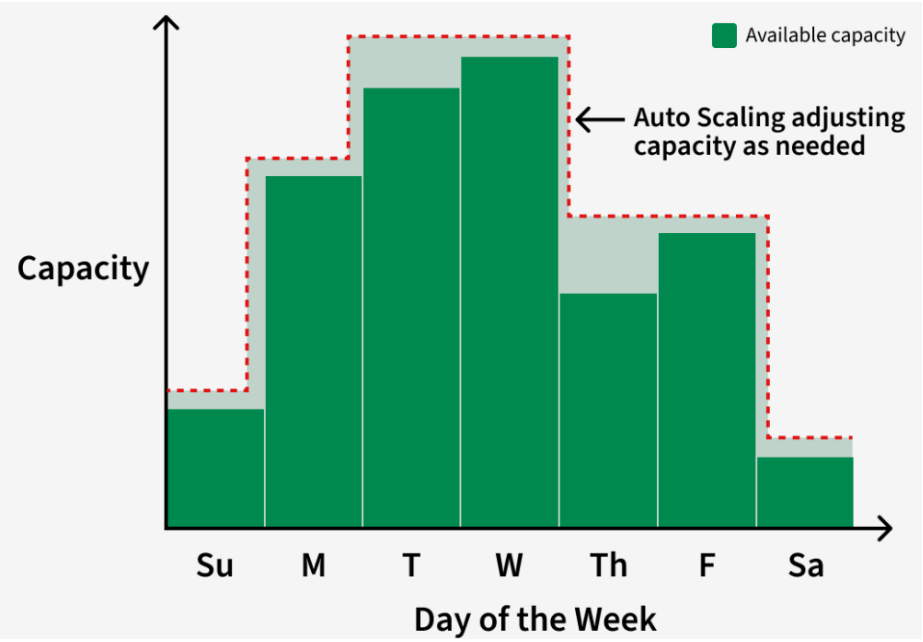
- **Maximum Capacity:** Maximum number of instances allowed to prevent extra costs.
- **Desired Capacity:** Number of instances that should run currently.



3. **Scaling Policies**

Scaling Polices decide when EC2 instances should be added or removed.

- Provisioning servers for peak traffic ensures demand is met but can lead to excess capacity and higher costs
- Allocating resources based on average demand reduces costs but may affect performance during spikes
- EC2 Auto Scaling automatically adds or removes instances based on real-time demand
- Uses EC2 instances to provide a cost-efficient architecture, charging only for resources actually used



Features of AWS Auto Scaling

Here are the some most important features of AWS Auto scaling

- **Dynamic Scaling:** Automatically increases or decreases EC2 instances based on real-time demand using metrics like CPU usage or request count.
- **Load Balancing:** Distributing incoming traffic across multiple EC2 instances to improve performance and availability using Elastic Load Balancing (ELB).
- **Multi-Availability Zone Deployment:** Launches instances across multiple Availability Zones to improve fault tolerance and maintain availability during AZ failures.
- **Containerization:** Supports containerized applications using Amazon ECS for easier deployment and management of Docker containers.

Types of AWS (Amazon Web Services) Autoscaling

AWS offers several ways to scale your infrastructure:

1. Predictive Scaling

- Uses machine learning to predict future traffic demand.
- Automatically scales resources before traffic increases.

2. Scheduled Scaling

- Used for predictable traffic patterns at specific times.
- Example: Increase instances before office hours and reduce them later.

3. Target Tracking Scaling

- Automatically maintains a target metric value such as CPU utilization.
- Example: Keeps CPU usage at 50% by adding or removing instances automatically.

4. Reactive Scaling

- Scales resources after traffic or workload changes are detected
- Example: Adds new instances automatically when CPU usage exceeds 70%

5. Vertical Scaling

- Increases or decreases the resources of a single EC2 instance
- Example: Upgrades an instance from t3.micro to t3.large for better performance

6. Horizontal Scaling

- Increases or decreases the number of EC2 instances
- Example: Adds more EC2 instances during high website traffic

Advanced Features

Mixed Instances Policy (Cost Optimization)

Mixed Instances Policy helps optimize cost and improve availability by using different EC2 instance types and pricing models.

- Supports both On-Demand and Spot Instances together
- Helps reduce infrastructure cost efficiently
- Allows multiple EC2 instance types
- Automatically selects the best available instance

Health Checks

Health Checks monitor instance and application health to maintain reliability and availability.

- EC2 Health Check monitors instance-level health
- Replaces failed or unhealthy EC2 instances
- ELB Health Check monitors application health
- Replaces instances returning application errors

Lifecycle Hooks

Lifecycle Hooks allow custom actions during instance launch or termination.

- Pauses instance launch or termination temporarily
- Allows custom actions during instance lifecycle
- Useful for installing software or running scripts
- Helps upload logs and gracefully close connections

Use Case

- **Automatic Scaling:** Application scaling can be done automatically based upon the incoming traffic if the load is increased then the application will scale up and the load decrease application will scale down automatically.
- **Schedule Scaling:** Based the data that previously available in at which particular point of time there going to be peak point and at which time there going to be less traffic we can schedule the auto scaling.

- **Integration:** You can integrate with other service in the AWS. Mainly the machine learning which will helps to predict the incoming traffic and can scale according to the traffic.

Working of AWS Auto Scaling

- Automatically adjusts the number of instances based on traffic or CPU load
- Monitors instances in an Auto Scaling group and maintains balanced performance
- Scales out when load increases and scales in when load decreases
- Replaces failed instances to maintain the desired capacity

What Is AWS CloudTrail?

AWS CloudTrail is an AWS service that helps you enable operational and risk auditing, governance, and compliance of your AWS account. Actions taken by a user, role, or an AWS service are recorded as events in CloudTrail. Events include actions taken in the AWS Management Console, AWS Command Line Interface, and AWS SDKs and APIs.

CloudTrail provides three ways to record events:

- **Event history** – The **Event history** provides a viewable, searchable, downloadable, and immutable record of the past 90 days of management events in an AWS Region. You can search events by filtering on a single attribute. You automatically have access to the **Event history** when you create your account. For more information, see Working with CloudTrail event history. There are no CloudTrail charges for viewing the **Event history**.

- **CloudTrail Lake** – AWS CloudTrail Lake is a managed data lake for capturing, storing, accessing, and analyzing user and API activity on AWS for audit and security purposes. CloudTrail Lake converts existing events in row-based JSON format to Apache ORC format. ORC is a columnar storage format that is optimized for fast retrieval of data. Events are aggregated into *event data stores*, which are immutable collections of events based on criteria that you select by applying advanced event selectors. You can keep the event data in an event data store for up to 3,653 days (about 10 years) if you choose the **One-year extendable retention pricing** option, or up to 2,557 days (about 7 years) if you choose the **Seven-year retention pricing** option. You can create an

event data store for a single AWS account or for multiple AWS accounts by using AWS Organizations. You can import any existing CloudTrail logs from your S3 buckets into an existing or new event data store. You can also visualize top CloudTrail event trends with Lake dashboards. For more information, see Working with AWS CloudTrail Lake.

CloudTrail Lake event data stores and queries incur charges. When you create an event data store, you choose the pricing option you want to use for the event data store. The pricing option determines the cost for ingesting and storing events, and the default and maximum retention period for the event data store. When you run queries in Lake, you pay based upon the amount of data scanned. For information about CloudTrail pricing and managing Lake costs, see AWS CloudTrail Pricing and Managing CloudTrail Lake costs.

- **Trails** – *Trails* capture a record of AWS activities, delivering and storing these events in an Amazon S3 bucket, with optional delivery to CloudWatch Logs and Amazon EventBridge. You can input these events into your security monitoring solutions. You can also use your own third-party solutions or solutions such as Amazon Athena to search and analyze your CloudTrail logs. You can create trails for a single AWS account or for multiple AWS accounts by using AWS Organizations. You can log Insights events to analyze your management events for anomalous behavior in API call rates and error rates. For more information, see Creating a trail for your AWS account.

  You can deliver one copy of your ongoing management events to your S3 bucket at no charge from CloudTrail by creating a trail, however, there are Amazon S3 storage charges. For more information about CloudTrail pricing, see AWS CloudTrail Pricing. For information about Amazon S3 pricing, see Amazon S3 Pricing.

Visibility into your AWS account activity is a key aspect of security and operational best practices. You can use CloudTrail to view, search, download, archive, analyze, and respond to account activity across your AWS infrastructure. You can identify who or what took which action, what resources were acted upon, when the event occurred, and other details to help you analyze and respond to activity in your AWS account.

You can integrate CloudTrail into applications using the API, automate trail or event data store creation for your organization, check the status of event data stores and trails you create, and control how users view CloudTrail events.

Accessing CloudTrail

You can work with CloudTrail in any of the following ways.

Topics

- CloudTrail console
- AWS CLI
- CloudTrail APIs
- AWS SDKs

CloudTrail console

Sign in to the AWS Management Console and open the CloudTrail console at https://console.aws.amazon.com/cloudtrail/.

The CloudTrail console provides a user interface for performing many CloudTrail tasks such as:

- Viewing recent events and event history for your AWS account.
- Downloading a filtered or complete file of the last 90 days of management events from **Event history**.
- Creating and editing CloudTrail trails.
- Creating and editing CloudTrail Lake event data stores.
- Running queries on event data stores.
- Configuring CloudTrail trails, including:
    - Selecting an Amazon S3 bucket for trails.
    - Setting a prefix.
    - Configuring delivery to CloudWatch Logs.
    - Using AWS KMS keys for encryption of trail data.
    - Enabling Amazon SNS notifications for log file delivery on trails.
    - Adding and managing tags for your trails.

- Configuring CloudTrail Lake event data stores, including:
    - Integrating event data stores with CloudTrail partners or with your own applications, to log events from sources outside of AWS.
    - Federating event data stores to run queries from Amazon Athena.
    - Using AWS KMS keys for encryption of event data store data.
    - Adding and managing tags for your event data stores.

For more information about the AWS Management Console, see AWS Management Console.

AWS CLI

The AWS Command Line Interface is a unified tool that you can use to interact with CloudTrail from the command line. For more information, see the AWS Command Line Interface User Guide. For a complete list of CloudTrail CLI commands, see cloudtrail and cloudtrail-data in the *AWS CLI Command Reference*.

CloudTrail APIs

In addition to the console and the CLI, you can also use the CloudTrail RESTful APIs to program CloudTrail directly. For more information, see the AWS CloudTrail API Reference and the CloudTrail-Data API Reference.

AWS SDKs

As an alternative to using the CloudTrail API, you can use one of the AWS SDKs. Each SDK consists of libraries and sample code for various programming languages and platforms. The SDKs provide a convenient way to create programmatic access to CloudTrail. For example, you can use the SDKs to sign requests cryptographically, manage errors, and retry requests automatically.

What is Amazon CloudWatch?

Amazon CloudWatch monitors your Amazon Web Services (AWS) resources and the applications you run on AWS in real time. You can use CloudWatch to collect and track metrics, which are variables you can measure for your resources and applications.

The CloudWatch home page automatically displays metrics about every AWS service you use. You can additionally create custom dashboards to display metrics about your custom applications, and display custom collections of metrics that you choose.

You can create alarms that watch metrics and send notifications or automatically make changes to the resources you are monitoring when a threshold is breached. For example, you can monitor the CPU usage and disk reads and writes of your Amazon EC2 instances and then use that data to determine whether you should launch additional instances to handle increased load. You can also use this data to stop under-used instances to save money.

With CloudWatch, you gain system-wide visibility into resource utilization, application performance, and operational health.

Accessing CloudWatch

You can access CloudWatch using any of the following methods:

- **Amazon CloudWatch console** – https://console.aws.amazon.com/cloudwatch/
- **AWS CLI** – For more information, see Getting Set Up with the AWS Command Line Interface in the *AWS Command Line Interface User Guide*.
- **CloudWatch API** – For more information, see the Amazon CloudWatch API Reference.
- **AWS SDKs** – For more information, see Tools for Amazon Web Services.

Related AWS services

The following services are used along with Amazon CloudWatch:

- **Amazon Simple Notification Service (Amazon SNS)** coordinates and manages the delivery or sending of messages to subscribing endpoints or clients. You use Amazon SNS with CloudWatch to send messages when an alarm threshold has been reached. For more information, see Setting up Amazon SNS notifications.
- **Amazon EC2 Auto Scaling** enables you to automatically launch or terminate Amazon EC2 instances based on user-defined policies, health status checks, and schedules. You can use a CloudWatch alarm with Amazon EC2 Auto Scaling to scale your EC2 instances based on demand. For more information, see Dynamic Scaling in the *Amazon EC2 Auto Scaling User Guide*.

- **AWS CloudTrail** enables you to monitor the calls made to the Amazon CloudWatch API for your account, including calls made by the AWS Management Console, AWS CLI, and other services. When CloudTrail logging is turned on, CloudWatch writes log files to the Amazon S3 bucket that you specified when you configured CloudTrail. For more information, see Logging Amazon CloudWatch API and console operations with AWS CloudTrail.
- **AWS Identity and Access Management (IAM)** is a web service that helps you securely control access to AWS resources for your users. Use IAM to control who can use your AWS resources (authentication) and what resources they can use in which ways (authorization). For more information, see Identity and access management for Amazon CloudWatch.

Elastic Load Balancing Documentation

What is an Application Load Balancer?

Elastic Load Balancing automatically distributes your incoming traffic across multiple targets, such as EC2 instances, containers, and IP addresses, in one or more Availability Zones. It monitors the health of its registered targets, and routes traffic only to the healthy targets. Elastic Load Balancing scales your load balancer as your incoming traffic changes over time. It can automatically scale to the vast majority of workloads.

Elastic Load Balancing supports the following load balancers: Application Load Balancers, Network Load Balancers, Gateway Load Balancers, and Classic Load Balancers. You can select the type of load balancer that best suits your needs.

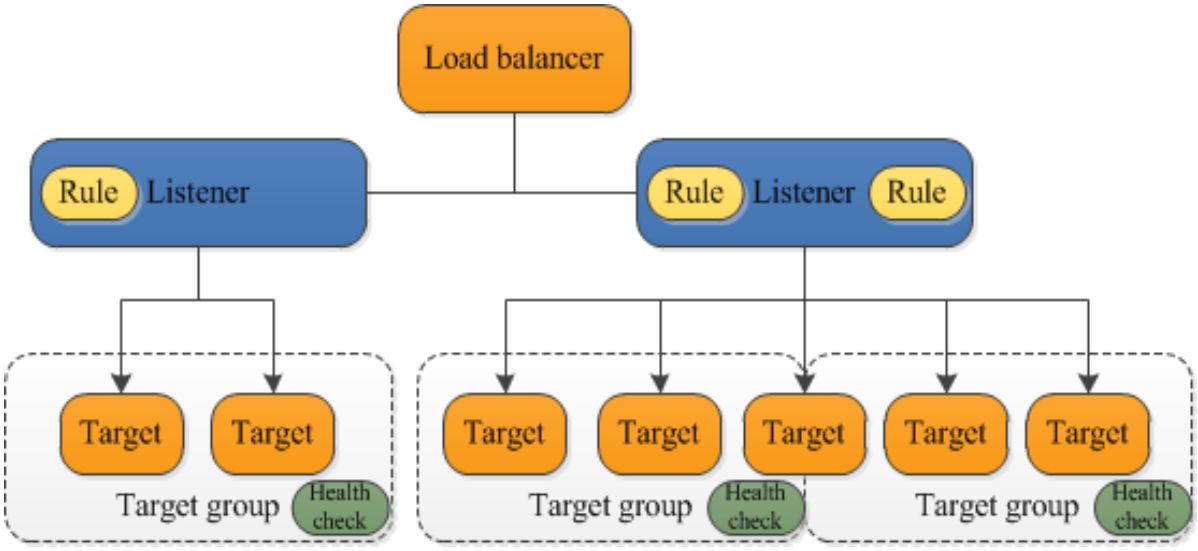
Application Load Balancer components

A *load balancer* serves as the single point of contact for clients. The load balancer distributes incoming application traffic across multiple targets, such as EC2 instances, in multiple Availability Zones. This increases the availability of your application. You add one or more listeners to your load balancer.

A *listener* checks for connection requests from clients, using the protocol and port that you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets. Each rule consists of a priority, one or more actions, and one or more conditions. When the conditions for a rule are met, then its

actions are performed. You must define a default rule for each listener, and you can optionally define additional rules.

Each *target group* routes requests to one or more registered targets, such as EC2 instances, using the protocol and port number that you specify. You can register a target with multiple target groups. You can configure health checks on a per target group basis. Health checks are performed on all targets registered to a target group that is specified in a listener rule for your load balancer.



Benefits of migrating from a Classic Load Balancer

Using an Application Load Balancer instead of a Classic Load Balancer has the following benefits:

- Support for Path conditions. You can configure rules for your listener that forward requests based on the URL in the request. This enables you to structure your application as smaller services, and route requests to the correct service based on the content of the URL.
- Support for Host conditions. You can configure rules for your listener that forward requests based on the host field in the HTTP header. This enables you to route requests to multiple domains using a single load balancer.
- Support for routing based on fields in the request, such as HTTP header conditions and methods, query parameters, and source IP addresses.

- Support for routing requests to multiple applications on a single EC2 instance. You can register an instance or IP address with multiple target groups, each on a different port.
- Support for redirecting requests from one URL to another.
- Support for returning a custom HTTP response.
- Support for registering targets by IP address, including targets outside the VPC for the load balancer.
- Support for registering Lambda functions as targets.
- Support for the load balancer to authenticate users of your applications through their corporate or social identities before routing requests.
- Support for containerized applications. Amazon Elastic Container Service (Amazon ECS) can select an unused port when scheduling a task and register the task with a target group using this port. This enables you to make efficient use of your clusters.
- Support for monitoring the health of each service independently, as health checks are defined at the target group level and many CloudWatch metrics are reported at the target group level. Attaching a target group to an Auto Scaling group enables you to scale each service dynamically based on demand.
- Access logs contain additional information and are stored in compressed format.
- Improved load balancer performance.

Create an Application Load Balancer

A load balancer takes requests from clients and distributes them across targets in a target group.

Before you begin, ensure that you have a virtual private cloud (VPC) with at least one public subnet in each of the zones used by your targets. For more information, see Subnets for your load balancer.

To create a load balancer using the AWS CLI, see Getting started with Application Load Balancers using the AWS CLI.

To create a load balancer using the AWS Management Console, complete the following tasks.

Tasks

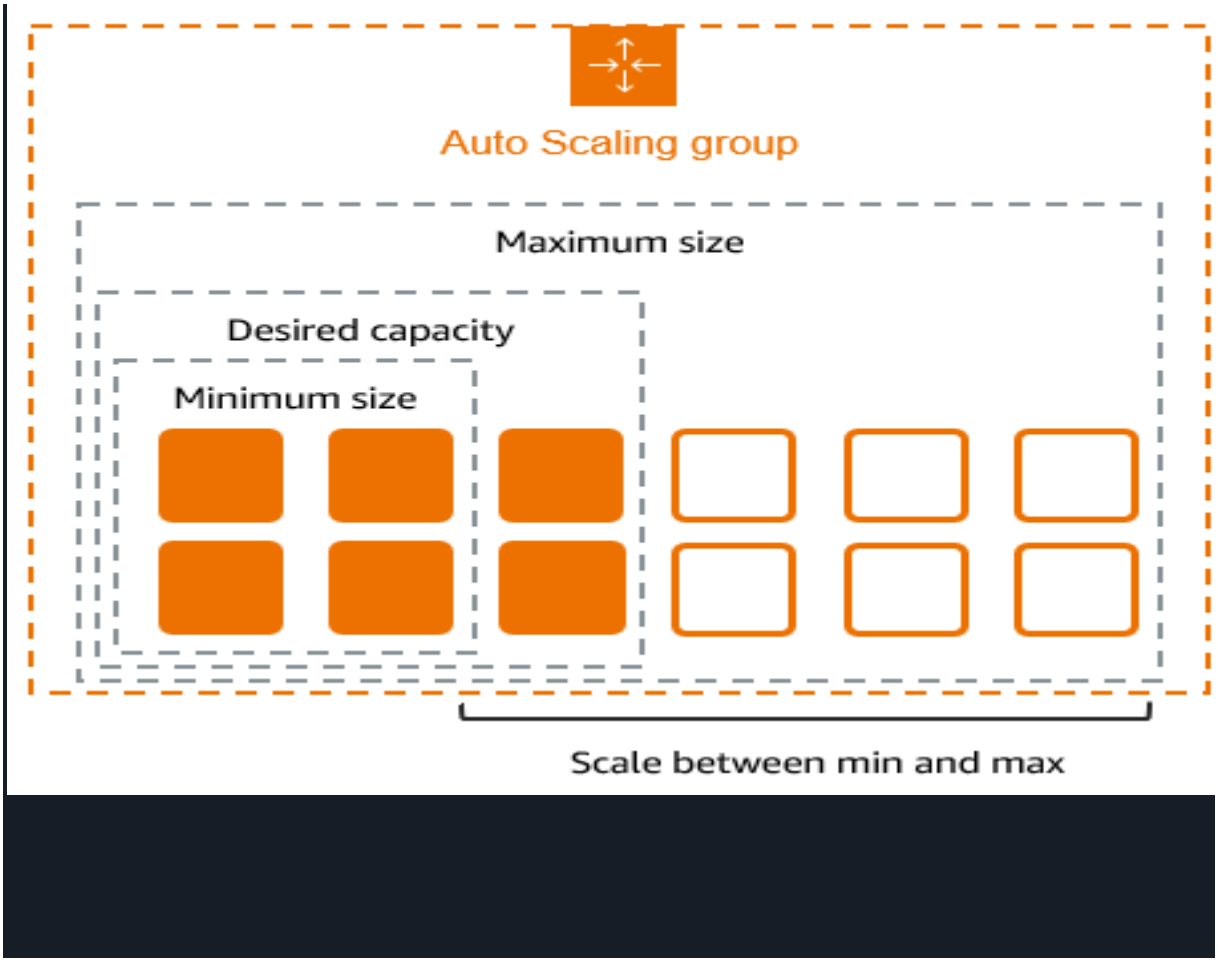
- Step 1: Configure a target group
- Step 2: Register targets
- Step 3: Configure a load balancer and a listener
- Step 4: Test the load balancer

Amazon EC2 Auto Scaling

Amazon EC2 Auto Scaling helps you ensure that you have the correct number of Amazon EC2 instances available to handle the load for your application. You create collections of EC2 instances, called *Auto Scaling groups*. You can specify the minimum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes below this size. You can specify the

maximum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes above this size. If you specify the desired capacity, either when you create the group or at any time thereafter, Amazon EC2 Auto Scaling ensures that your group has this many instances. If you specify scaling policies, then Amazon EC2 Auto Scaling can launch or terminate instances as demand on your application increases or decreases.

For example, the following Auto Scaling group has a minimum size of four instances, a desired capacity of six instances, and a maximum size of twelve instances. The scaling policies that you define adjust the number of instances, within your minimum and maximum number of instances, based on the criteria that you specify.



Features of Amazon EC2 Auto Scaling

With Amazon EC2 Auto Scaling, your EC2 instances are organized into Auto Scaling groups so that they can be treated as a logical unit for the purposes of scaling and management. Auto Scaling groups use launch templates (or launch configurations) as configuration templates for their EC2 instances.

The following are key features of Amazon EC2 Auto Scaling:

Monitoring the health of running instances

Amazon EC2 Auto Scaling automatically monitors the health and availability of your instances using EC2 health checks and replaces terminated or impaired instances to maintain your desired capacity.

Custom health checks

In addition to the built-in health checks, you can define custom health checks that are specific to your application to verify that it's responding as expected. If an instance fails your custom health check, it's automatically replaced to maintain your desired capacity.

Balancing capacity across Availability Zones

You can specify multiple Availability Zones for your Auto Scaling group, and Amazon EC2 Auto Scaling balances your instances evenly across the Availability Zones as the group scales. This provides high availability and resiliency by protecting your applications from failures in a single location.

Multiple instance types and purchase options

Within a single Auto Scaling group, you can launch multiple instance types and purchase options (Spot and On-Demand Instances), allowing you to optimize costs

through Spot Instance usage. You can also take advantage of Reserved Instance and Savings Plans discounts by using them in conjunction with On-Demand Instances in the group.

Automated replacement of Spot Instances

If your group includes Spot Instances, Amazon EC2 Auto Scaling can automatically request replacement Spot capacity if your Spot Instances are interrupted. Through Capacity Rebalancing, Amazon EC2 Auto Scaling can also monitor and proactively replace your Spot Instances that are at an elevated risk of interruption.

Load balancing

You can use Elastic Load Balancing load balancing and health checks to ensure an even distribution of application traffic to your healthy instances. Whenever instances are launched or terminated, Amazon EC2 Auto Scaling automatically registers and deregisters the instances from the load balancer.

Scalability

Amazon EC2 Auto Scaling also provides several ways for you to scale your Auto Scaling groups. Using auto scaling allows you to maintain application availability and reduce costs by adding capacity to handle peak loads and removing capacity when demand is lower. You can also manually adjust the size of your Auto Scaling group as needed.

Instance refresh

The instance refresh feature provides a mechanism to update instances in a rolling fashion when you update your AMI or launch template. You can also use a phased approach, known as a canary deployment, to test a new AMI

or launch template on a small set of instances before rolling it out to the whole group.

Lifecycle hooks

Lifecycle hooks are useful for defining custom actions that are invoked as new instances launch or before instances are terminated. This feature is particularly useful for building event-driven architectures, but it also helps you manage instances through their lifecycle.

Support for stateful workloads

Lifecycle hooks also offer a mechanism for persisting state on shut down. To ensure continuity for stateful applications, you can also use scale-in protection or custom termination policies to prevent instances with long-running processes from terminating early.

For more information about the benefits of Amazon EC2 Auto Scaling, see Auto Scaling benefits for application architecture.

Introduction to Amazon CloudWatch

Amazon CloudWatch is a monitoring and observability service used to monitor applications and infrastructure. It helps DevOps engineers, developers, SREs, and IT teams track system performance using metrics, logs, and events. CloudWatch also helps detect issues, respond to changes, and optimize resource utilization through a unified monitoring dashboard.

Core Architecture

CloudWatch is built around three main components:

Metrics

Metrics are numerical values collected over time to monitor resources and applications.

- **Example:** CPU Utilization is 85% at 10:00 AM.
- AWS services like EC2, RDS, and Lambda automatically send metrics to CloudWatch.
- Users can also create custom metrics from their own applications.

Logs

Logs are text records that store detailed information about events and activities.

- **Example:** Lambda function errors or web server access logs.
- Logs are stored in Log Groups and Log Streams for better organization.

Events (now EventBridge)

Events track changes that happen in AWS resources and services.

- **Example:** An EC2 instance changing from running to stopped state.
- These events can trigger automated actions and workflows.

Features

Metrics

- Metrics are time-ordered data points published to CloudWatch.
- Each data point is associated with a timestamp.
- Metrics help monitor the performance of AWS resources and applications over time.
- AWS services automatically send metrics to CloudWatch, and users can also create custom metrics.
- Metric Math allows users to combine multiple metrics and create new calculations or time series.

Dimensions

- Dimensions are name/value pairs used to uniquely identify a metric.
- Adding a new dimension creates a different variation of the metric.
- They help filter and organize monitoring data.

Statistics

- Statistics are aggregated metric values calculated over a period of time.
- The few available statistics on CloudWatch are maximum, minimum, sum, average, and sample count.

Alarm

- It is used to automatically initiate actions on our behalf.
- It watches a single metric over a specified time period and performs one or more specified actions based on the value of the metric.
- The estimated AWS charges can also be monitored using the alarm.

Percentiles

- It represents the relative weightage of the data in a dataset.
- It helps the user to get a better understanding of the distribution of metric data.

CloudWatch dashboard

- A user-friendly CloudWatch console is available which is used for monitoring resources in a single view.
- There is no limit on the number of CloudWatch dashboards you can create.
- These dashboards are global and not region-specific.

CloudWatch agent

- The CloudWatch Agent is required only for collecting system-level metrics and custom logs that are not available by default.
- It collects logs and system-level metrics from EC2 instances and on-premises servers.

EventBridge (formerly CloudWatch Events)

- Amazon EventBridge monitors events generated by AWS services.
- Events include changes like an EC2 instance moving from running to stopped state.
- Rules are created to match specific event patterns.
- This helps automate notifications, workflows, and serverless actions.

CloudWatch logs

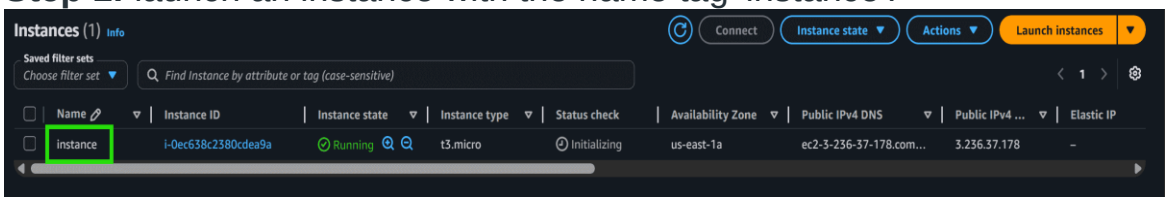
- Amazon CloudWatch logs enable you to store, monitor, and access files from AWS resources like Amazon EC2 instances, Route53, etc.
- It also helps you to troubleshoot your system errors and maintain the logs in highly durable storage.
- It also creates log of information about the DNS queries that Route 53 receives.

Getting started with Amazon CloudWatch

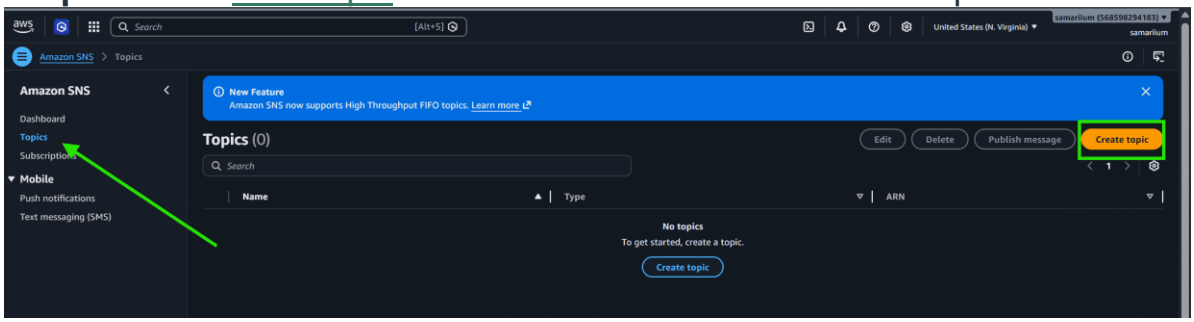
Notifying *gfg* website management team when the instance on which *gfg* website is hosted stops Whenever the CPU utilization of instance (on which GeeksForGeeks website is hosted) goes above 80%, CloudWatch event is triggered. This CloudWatch event then activates the SNS topic which sends the alert email to the attached *gfg* subscribers.

Create an SNS Topic

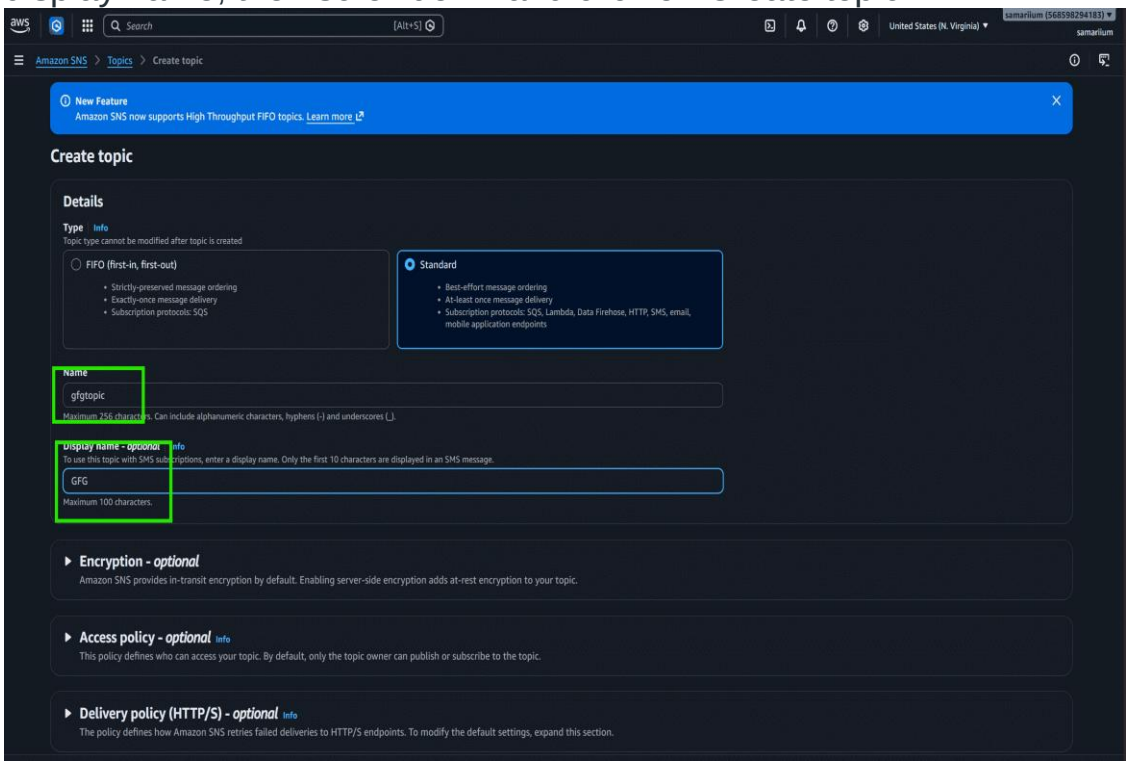
Step 1: launch an instance with the name tag 'instance'.



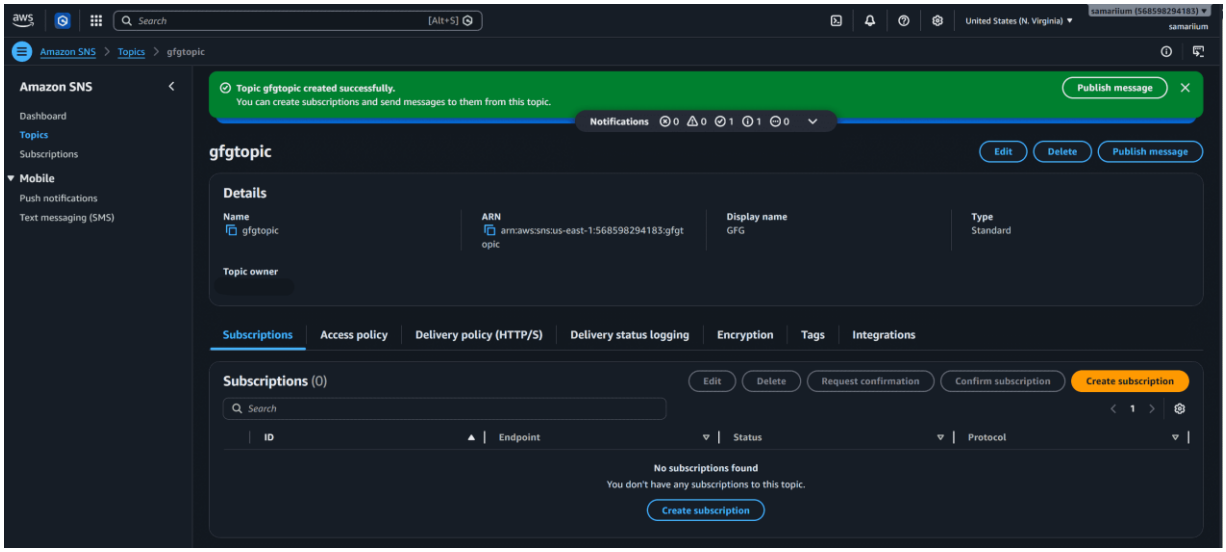
Step 2: Go to SNS topic dashboard and click on create a topic.



Step 3: You will be redirected to the dashboard. Enter the topic name and display name, then scroll down and click on Create topic.

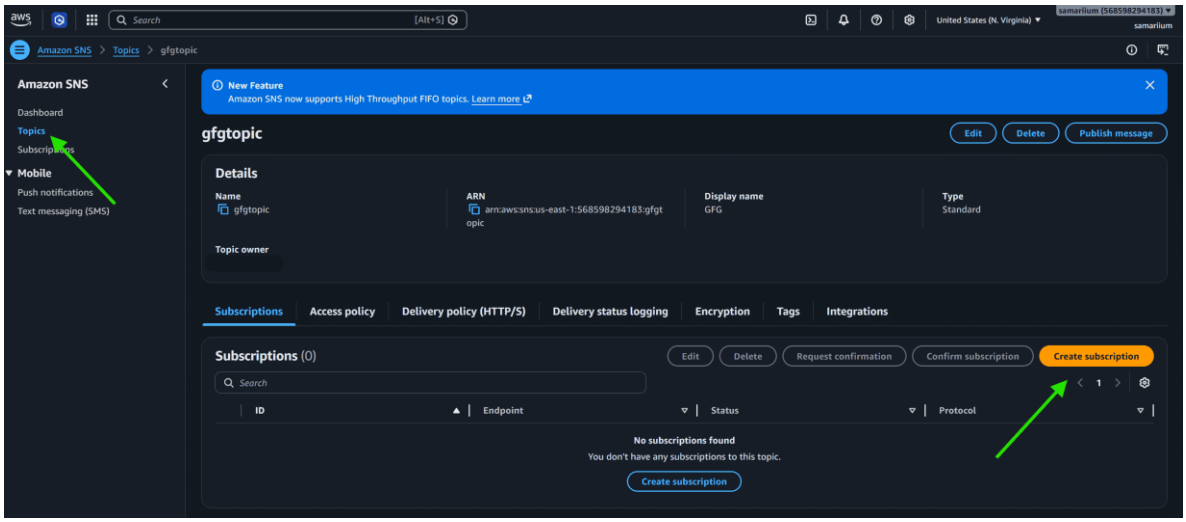


Step 4: The SNS topic is created successfully.

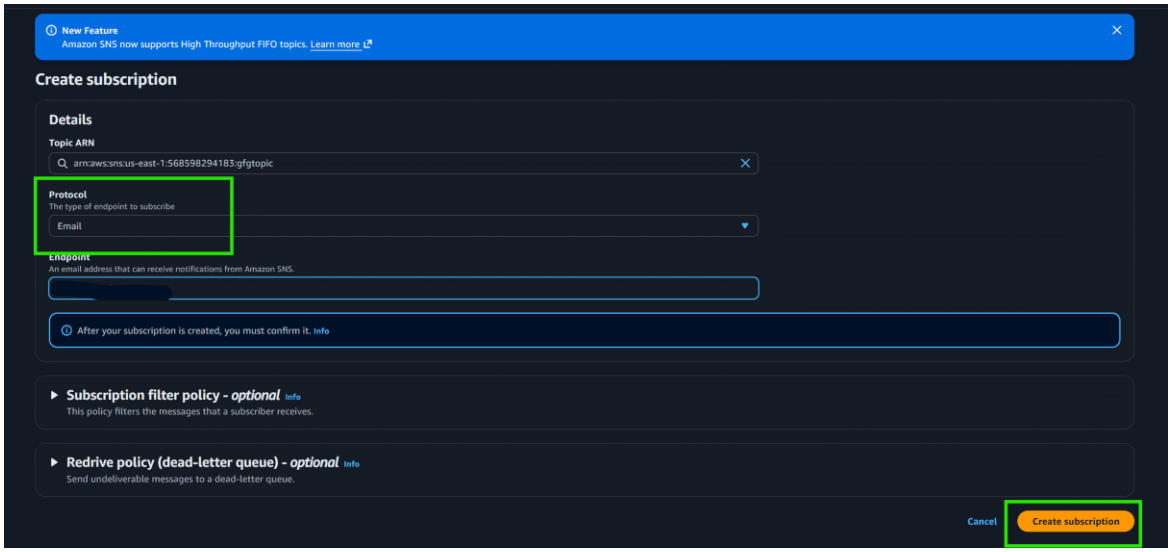


Add Subscribers

Step 1: Go to the SNS Topics dashboard, click on the gfgtopic link, and under the Subscriptions section, click on Create subscription.



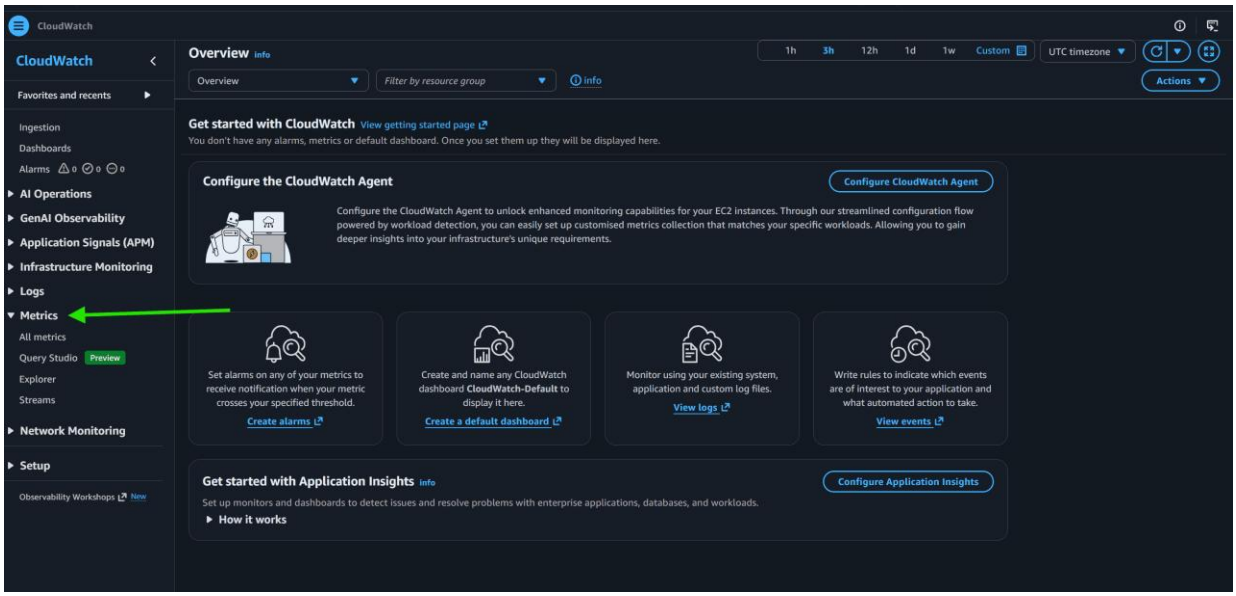
Step 2: Select Email as protocol and specify the email address of subscribers in Endpoint. Click on create the subscription. Now Go to the mailbox of the specified email id and click on Subscription confirmed.



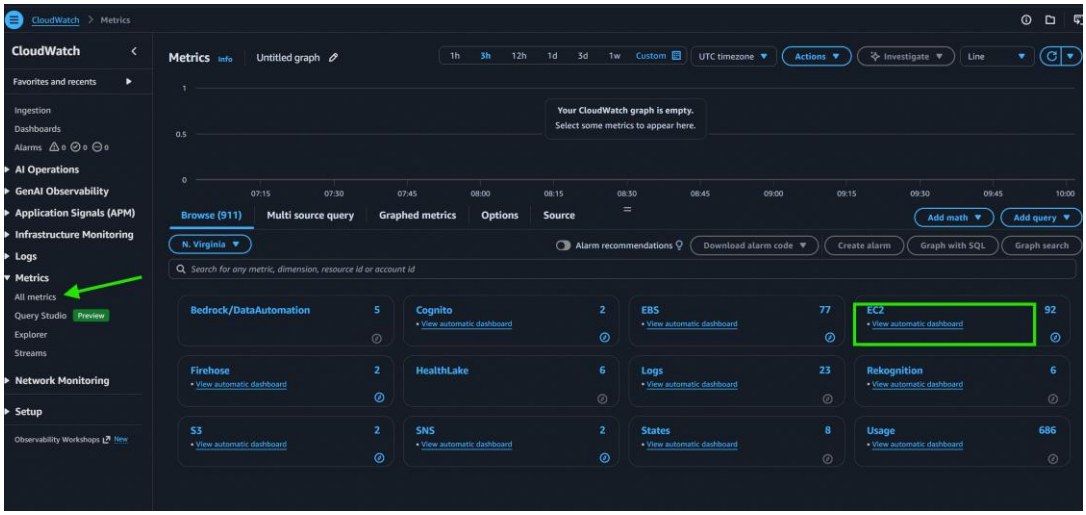
Craft the CloudWatch Alarm

Step 1: Go to the CloudWatch dashboard on the AWS management console.

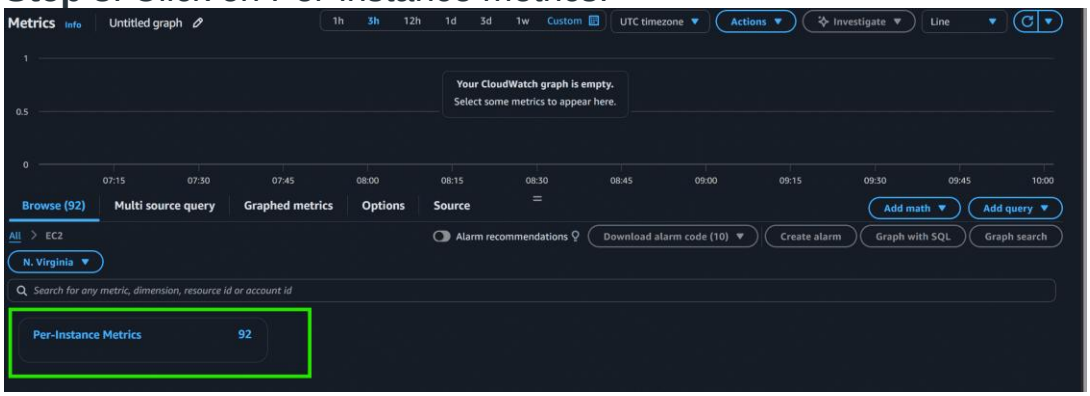
Click on Metrics in the left pane.



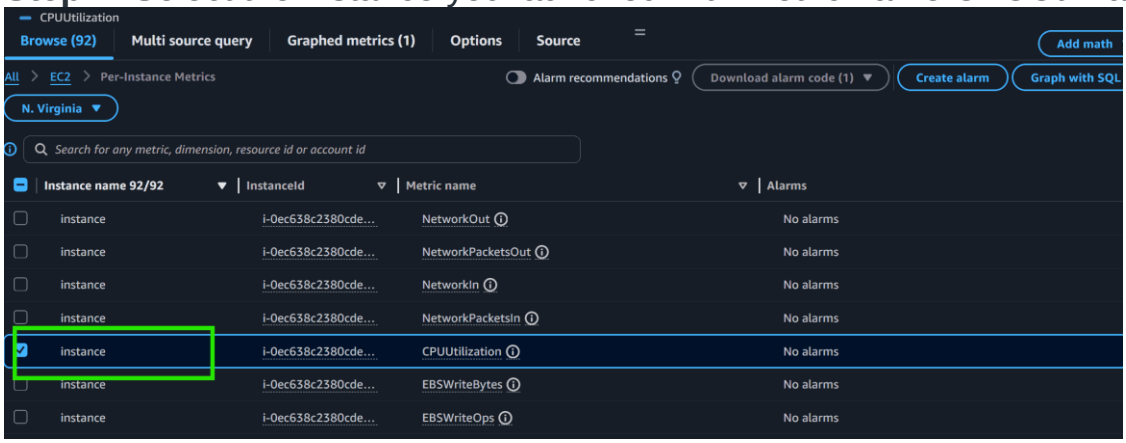
Step 2: In All metrics section click on EC2



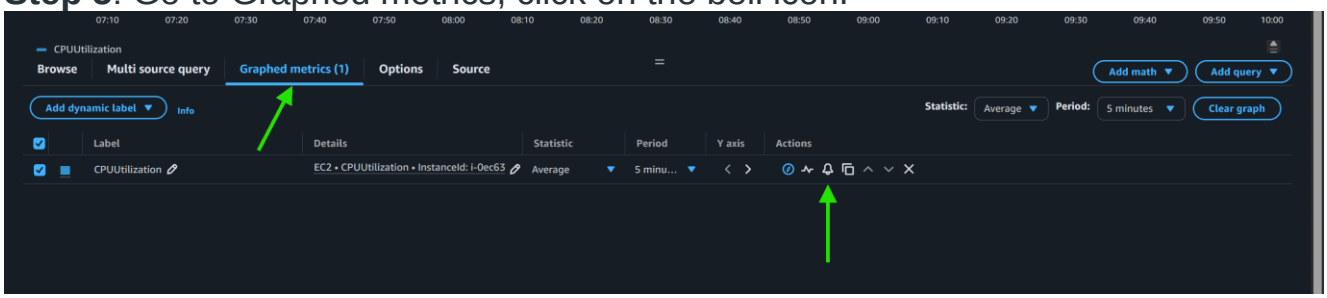
Step 3: Click on Per-instance metrics.



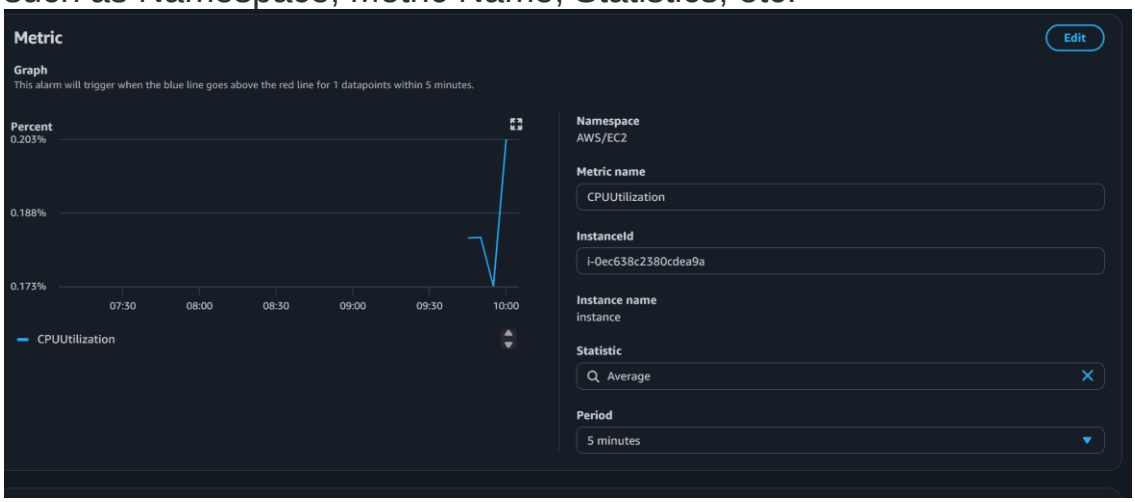
Step 4: Select the instance you launched with Metric name CPUUtilization.



Step 5: Go to Graphed metrics, click on the bell icon.

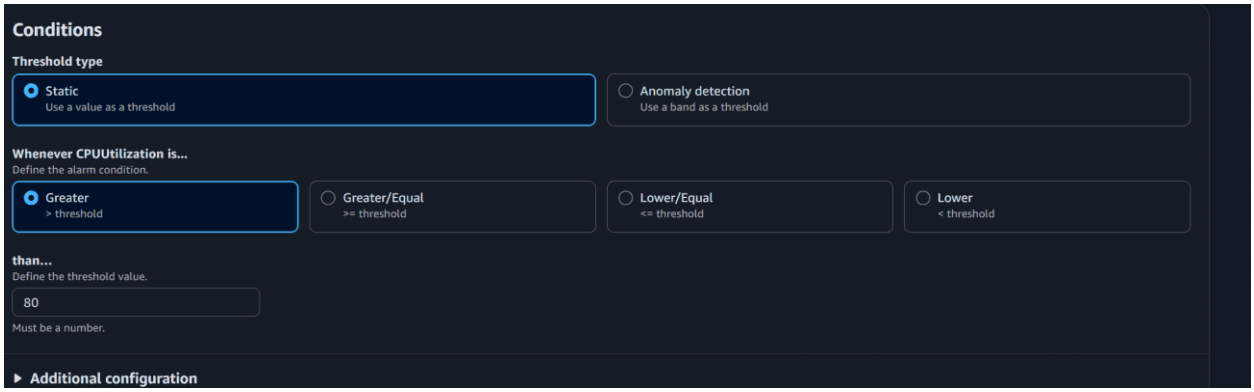


Step 6: This dashboard shows the components of Amazon CloudWatch such as Namespace, Metric Name, Statistics, etc.



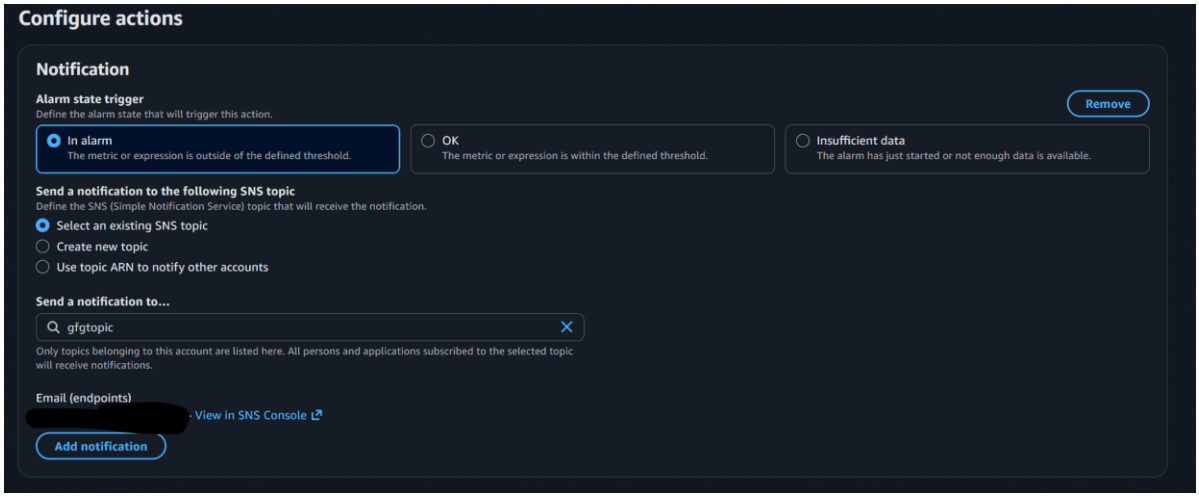
Set the Threshold

Step 1: Select the greater threshold. Also, specify the amount(i.e. 80) of the threshold value. Click on Next.



Connecting

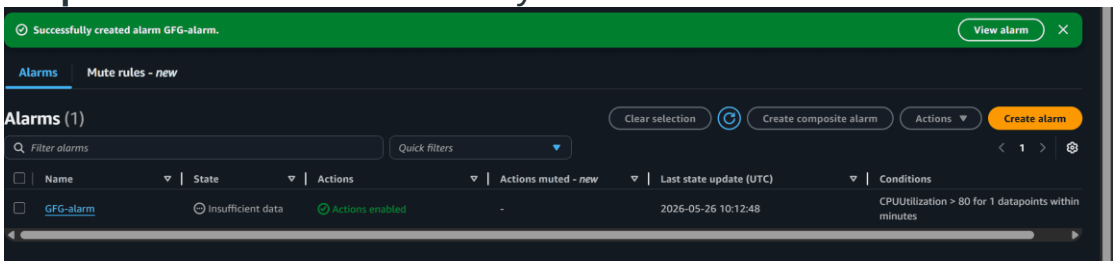
Step 1: Click on Select an existing SNS topic, also mention the name of the SNS topic you created now.



Step 2: Specify the name of alarm and description which is completely optional. Click on Next and then click on Create alarm.



Step 3: The alarm is successfully created.



Step 4: You can see the graph which notifies whenever CPU utilization goes above 80%.



Creating a dashboard in Amazon CloudWatch

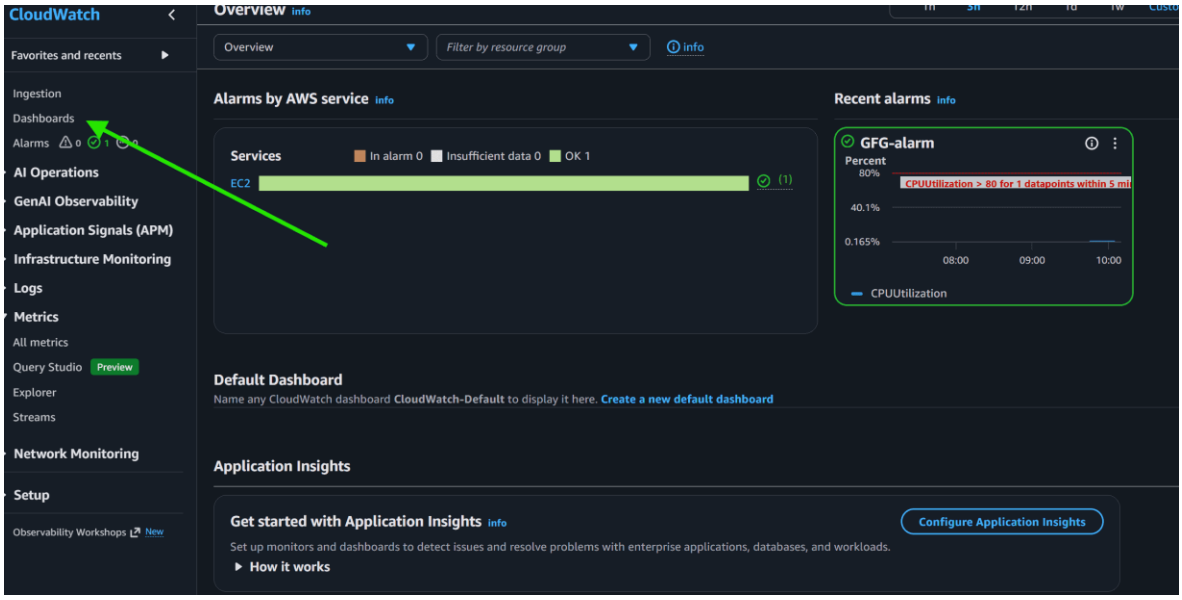
Creating a dashboard in Amazon CloudWatch is a straightforward process that allows you to visualize and monitor metrics and logs in a centralized and customizable view. Here's a step-by-step guide on how to create a dashboard in Amazon CloudWatch:

Step 1: Access Amazon CloudWatch Console

- Open the Amazon CloudWatch Console.

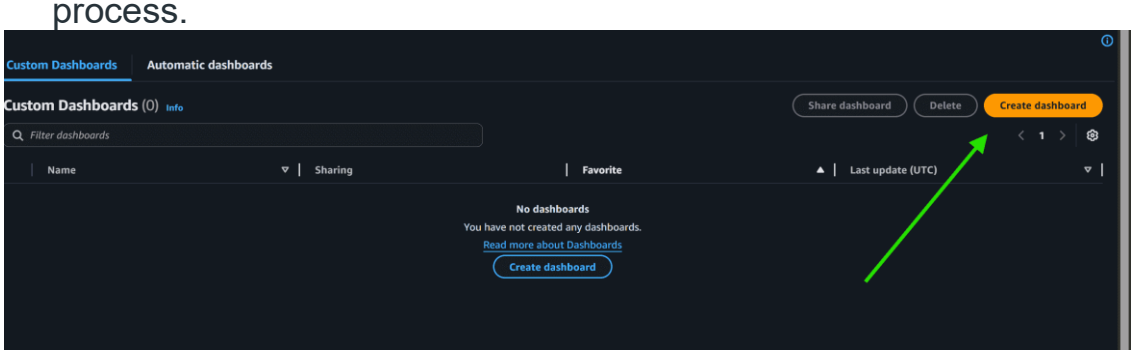
Step 2: Navigate to Dashboards

- In the CloudWatch Console, locate the "Dashboards" section in the left navigation pane.



Step 3: Click "Create Dashboard"

- Click the "Create dashboard" button to start the dashboard creation process.

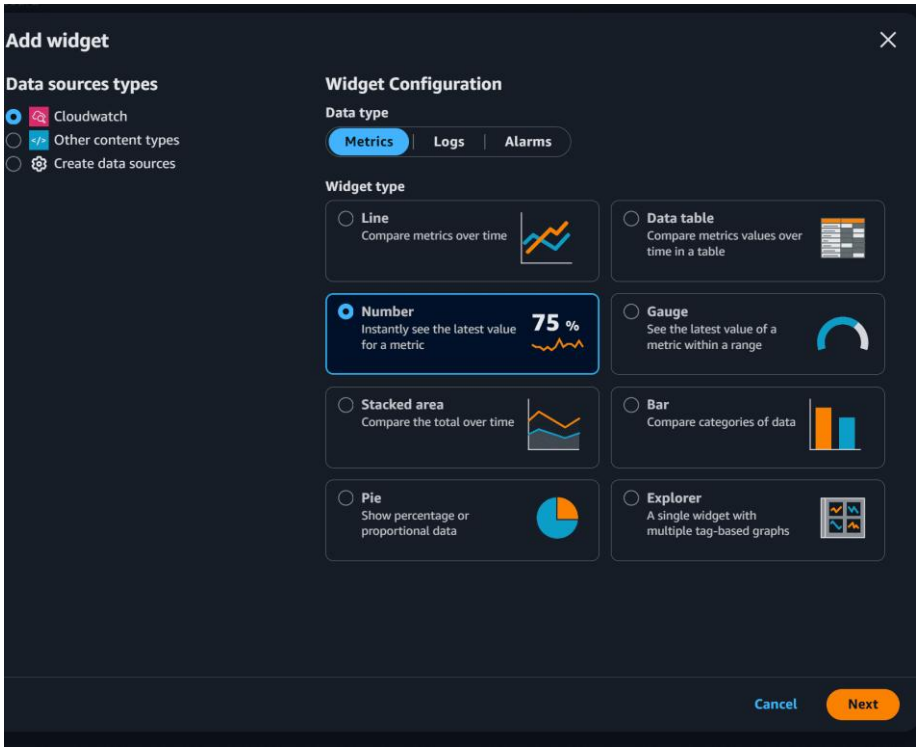


Step 4: Enter Dashboard Details

- Provide a name for your dashboard in the "Dashboard name" field. Optionally, you can also add a description.

Step 5: Add a Number Widget for CPU Utilization

- Click on Add widget.
- Select the Number widget type.
- Enter the widget name as CPU Utilization.
- Choose EC2 under Namespace.
- Select Per-Instance Metrics.
- Choose CPUUtilization as the metric.
- Configure additional settings if needed and save the widget.



Step 6: Save the Dashboard

- Once you've configured the widget, click the "Create dashboard" or "Save dashboard" button to save your changes.

Step 7: View the Dashboard

- Your dashboard is now created with a Number widget displaying the CPU utilization of your EC2 instances. You can view it in the CloudWatch Console.

CloudWatch vs. CloudTrail

These two services are often confused, but they serve very different purposes.

Feature	Amazon CloudWatch	AWS CloudTrail
Focus	Performance & Monitoring.	Auditing & Compliance.
Purpose	Monitors applications and infrastructure performance	Tracks user activity and API calls in AWS
Data Type	Metrics & Logs.	API Activity History.
Example	90% CPU Utilization.	User Samar called "TerminateInstance" at 2:00 PM.

Benefits

- Provides a centralized dashboard to monitor application and infrastructure data.
- Helps reduce operational costs by using alarms and automated actions.
- Optimizes applications and AWS resources using metrics and logs analysis.

- Gives detailed insights such as CPU usage, memory utilization, and capacity utilization.
- Makes it easy to compare monitoring data across different AWS services.

Drawbacks

- Can become expensive for large-scale monitoring and logging.
- Handling very large amounts of log data during high traffic can be difficult.
- Monitoring and logging activities may consume additional system resources.
- Integration with other AWS services and third-party tools can sometimes be complex.
- Setup and management may be challenging for beginners or users unfamiliar with cloud systems.